

Cloud Operations FS26

Exam Summary

Björn Seger

29.06.2026

Contents

1	Introduction to Cloud Operations	1
1.1	DevOps	1
1.2	The DevOps Lifecycle	1
1.3	Continuous Lifecycle	5
1.4	Platform Engineering	5
1.5	Key Takeaways	6
2	CI/CD with GitLab	7
2.0.1	Benefits	7
2.1	Concepts	7
2.2	GitLab CI Pipeline Basics	8
2.2.1	Typical Stages	8
3	Infrastructure Provisioning (IaC)	11
3.0.1	Benefits	11
3.1	Imperative vs. Declarative	11
3.2	Terraform/OpenTofu	11
3.2.1	HCL & File Conventions	12
3.2.2	File Naming Best Practices	12
3.2.3	Terraform Core Concepts	12
3.2.4	The State File	15
3.2.5	Backends: Team Collaboration	17
3.2.6	Terraform Workflow & Provisioners	18
4	Configuration Management	20
4.1	What is Ansible?	20
4.1.1	Terraform vs. Ansible	20
4.2	Ansible Architecture & Connection Model	21
4.2.1	Inventory	21
4.3	Plays & Playbooks	22
4.4	Core Constructs	22
4.5	Templating (Jinja2)	23
4.5.1	Filters	24
4.6	Tags	24
4.7	Collections & Roles	25
4.8	Best Practices	25
5	Kubernetes Object Management	27
5.1	Foundations	27
5.2	Workloads	28
5.3	kubectl Usage Patterns	28
5.4	Managing Deployments & Rolling Updates	29
5.5	Namespaces & Resource Management	29
5.5.1	Resource governance	30
5.6	Managing Scheduling	30
5.6.1	Filtering	30
5.6.2	Scoring	30
5.6.3	Taints and Tolerations	31
5.7	Networking	31
5.7.1	Accessing Pods	31
5.8	Storage	32
5.9	ConfigMaps and Secrets	32
5.10	Probes	33
6	Kubernetes Resource Packaging	35

6.1	Helm	35
6.1.1	Core Concepts	35
6.1.2	Helm Charts	35
6.1.3	Templates	36
6.1.4	Helm lifecycle commands	37
6.2	Kustomize	38
6.2.1	ConfigMaps/Secrets in Kustomize	40
6.3	Comparison & Best Practices	40
6.3.1	Best practices	40
6.3.2	Common pitfalls	40
7	Kubernetes Security — Cilium Networking and Policies	42
7.1	Cilium Networking	43
7.2	Cilium Network Core Concepts	44
7.3	Policy Definition	46
7.3.1	Policy Layers	46
7.3.2	Multiple Rules per Policy	46
7.3.3	Advanced Endpoint Selectors	47
7.3.4	toEndpoints/fromEndpoints	47
7.3.5	toCIDR/fromCIDR and toCIDRSet/fromCIDRSet	48
7.3.6	toFQDNs L3 Policies	49
7.3.7	toFQDNs DNS proxy requirement	49
7.3.8	toEntities/fromEntities + Reserved L3 Identities	49
7.3.9	toPorts	50
7.3.10	Port Ranges	50
7.3.11	L7 Policy Example (HTTP)	51
7.4	Common patterns	52
7.4.1	Default Deny Policy	52
7.4.2	L7 Visibility Policies	53
7.5	Getting Started with Cilium Network Policies & Hubble	54
7.5.1	Common Gotchas	54
8	Service Mesh	55
8.1	Core components	55
8.2	Sidecarless Service Mesh	56
8.2.1	Cilium VS Istio Approach	56
8.3	Traffic management / deployment strategies	56
8.3.1	GAMMA	56
8.3.2	Rolling Deployment	57
8.3.3	Recreate / Big-bang Deployment	57
8.3.4	Canary Deployment	57
8.3.5	Blue/Green Deployment	57
8.3.6	Dark Launching / Shadow Deployment	57
8.4	Security	57
9	Kubernetes Observability	59
9.1	Monitoring with Prometheus	59
9.1.1	Architecture	60
9.1.2	PromQL	60
9.1.3	Scaling	61
9.2	Visualization & Alerting (Grafana)	61
9.2.1	Querying	61
9.2.2	Dashboards	62
9.2.3	Alerting	62
9.3	Logging with Loki	63
9.3.1	Kubernetes logging approaches	63

9.3.2	Loki	63
9.3.3	Alloy	65
9.4	Tracing	66
9.5	Wrap-Up	66
9.5.1	Monitoring and logging — best practices	66
9.5.2	Common pitfalls	66
9.5.3	Alternative Stacks	66
10	GitOps	67
10.1	Principles	67
10.2	Traditional CI/CD vs. GitOps	67
10.3	Push vs. pull	68
10.4	Argo CD	68
10.5	Flux CD	69
10.6	GitOps Workflow Patterns	70
10.7	Secrets Management	70

1. Introduction to Cloud Operations

Learning Objectives

- Understand the concept of DevOps, CI/CD, and Platform Engineering
- Describe all stages of the DevOps cycle
- Associate activities with DevOps cycle stages
- Understand the relationship between DevOps, CI/CD, and Platform Engineering

1.1 DevOps

DevOps

A set of practices combining software development (Dev) and IT operations (Ops) to shorten the development lifecycle and deliver high-quality software continuously.

1.2 The DevOps Lifecycle

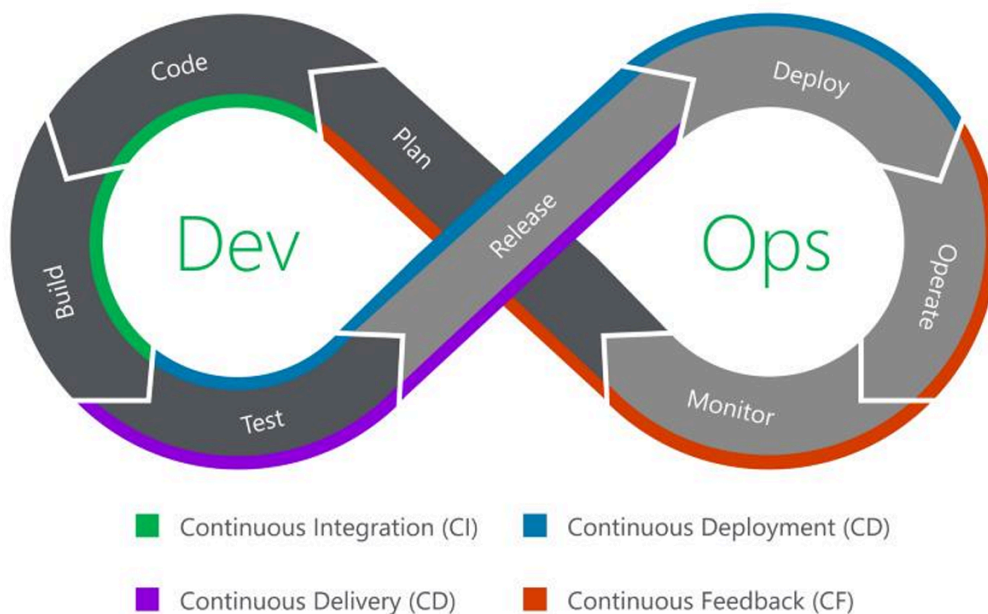


Figure 1: The DevOps Lifecycle in Phases

1

Plan

Continuous Feedback

Key Activities

- Requirements, goals, roadmap & stories creation
- Architecture sketch
- Security checks plan
- Metrics & Alerts analysis
- Risk & rollback strategy

Tools

Jira · Asana · GitHub Issues

2

Code

Continuous Integration

Key Activities

- Write & refactor code
- Local builds
- Local tests
- Peer review & gated merges

Tools

GitLab · GitHub · Bitbucket

3

Build

Continuous Integration

Key Activities

- Compile source codes / generate binaries
- Create container images
- Publish artifacts
- Generate versioned release packages
- Store build metadata

Tools

GitLab CI/CD, GitHub Actions, Jenkins, Azure Pipelines · Maven, Gradle, Make, Taskfile · Docker, Podman, Buildah · Artifactory, GitLab/GitHub Package · Quay / Google / GitLab / GitHub Image Registry

4

Test

Continuous Delivery

Continuous Deployment

Key Activities

- Execute automated test suites on each commit
- Run UI / end-to-end tests against a test environment
- Conduct & perform load testing
- Carry out security / vulnerability scans on built artefacts
- Measure code-coverage and enforce quality gates
- Collect and publish test reports, logs, and metrics for traceability

Tools

GitLab CI/CD, GitHub Actions, Jenkins, CircleCI · JUnit / TestNG / pytest / Mocha · Selenium / Cypress / Playwright · JMeter / Gatling / Locust / k6 · Snyk / Trivy · Codecov / SonarQube / SonarCloud

5

Release

Continuous Delivery

Continuous Deployment

Key Activities

- Promote the verified artefact to a release repo
- Assign an immutable version
- Run final pre-deployment sanity checks
- Publish release notes and notify stakeholders
- Store release metadata (buildID, commit SHA, artefact hash) for traceability

Tools

GitLab CI/CD, GitHub Actions, Azure Pipelines, CircleCI, Jenkins · Artifact repositories: GitLab Package Registry, GitHub Packages, Artifactory · Signing/verification: GnuPG, Cosign, Notary

6

Deploy

Continuous Deployment

Key Activities

- Pull released artefact into the target environment
- Apply the deployment manifest
- Execute a deployment strategy
- Perform post deployment validation
- Provide / Trigger rollback if validation fails
- Communicate deployment status to the team(if appropriate also to customer)
- Archive the deployed version and config for audit purposes

Tools

ArgoCD, Flux, AWS CodeDeploy, Google Cloud Deploy, Azure DevOps Release · Kubernetes, Kustomize, Helm · Terraform / OpenTofu, Pulumi · Ansible, Chef, Puppet · Canary/feature-flag controllers: Flagger, LaunchDarkly, Unleash

7

Operate

Continuous Feedback

Key Activities

- Document, troubleshoot and resolve issues
- Scaling resources, applying patches, rotating secrets
- Manage configuration drift detection and enforce desired-state
- Back up critical data and verify restore procedure regularly
- Ensure compliance reporting for regulatory requirements

Tools

ServiceNow, PagerDuty, Opsgenie, VictorOps · HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, 1Password · Velero, Restic, AWS Backup, Azure Backup, BorgBackup, rsnapshot · Kubernetes Horizontal/Vertical Pod Autoscaler, AWS Auto Scaling Groups, Google Cloud Autoscaler, Azure VM Scale Sets · OpenSCAP, Falco, Aqua Security, Prisma Cloud, AWS Config, Azure Policy

8

Monitor

Continuous Feedback

Key Activities

- Define service-level indicator (SLIs), service-level objectives
- Build real-time dashboards
- Configure and periodically check alerting rules
- Run health checks to verify end-to-end functionality
- Correlate logs, metrics, and traces

Tools

Prometheus, InfluxDB, Graphite, Datadog Metrics, NewRelic Metrics · Grafana, Kibana, Datadog Dashboard, New Relic One · Jaeger, Zipkin, Lightstep, Dynatrace · ElasticStack (ELK/EFK), Loki+ Grafana, Splunk, Graylog · Pingdom, UptimeRobot, Grafana Synthetic Monitoring, NewRelic Synthetics · PrometheusAlert-manager, Grafana Alerting

- **Dev side:** Plan → Code → Build → Test
- **Ops side:** Deploy → Operate → Monitor
- **Release** is the bridge connecting both halves

1.3 Continuous Lifecycle

Continuous Integration (CI)

Practice of frequently merging code changes into a shared repository, with automated build and test on every change.

Entails:

- **Continuous Development** = Plan + Code
- **Continuous Integration** (narrower) = Code + Build
- **Continuous Testing** = Test

Continuous Deployment / Delivery (CD)

Automatically deploying every change that passes CI to staging/production.

Entails:

- **Continuous Delivery** = Test + Release
 - code is automatically validated and packaged
- **Continuous Deployment**: Test + Release + Deploy
 - goes one step further and automatically deploys every validated change to production

Continuous Deployment Strategies

- **Rolling deployment**: Update gradually to minimize downtime
- **Blue-green**: Two different environments for fast switching
- **Canary release**: Specify a user group to deploy to first
- **Feature flags**: Deploy but activate later
- **Dark launching**: Use real user traffic but hide it from them

Continuous Feedback

Practice of feeding insights from the Operate and Monitor stages (incidents, metrics, usage data) back into Planning, closing the DevOps loop so each cycle informs the next

Operate → Monitor → Plan

1.4 Platform Engineering

Platform Engineering

A cultural mindset focused on designing, building, and operating internal developer platforms (IDPs)
- essentially treating infrastructure/tooling as a product for internal developers to use.

Core Responsibilities

- **Enable self-service**: Let developers provision what they need without filing tickets to Ops
- **Standardize runtimes & tooling**: Consistent stacks across teams

- **Abstract infrastructure:** Hide underlying complexity (clouds, clusters, networking) behind simple interfaces
- **Enforce compliance & security:** Bake policy/guardrails into the platform itself
- **Provide observability:** Built-in monitoring/logging by default

Typical Building Blocks (bottom-up stack):

1. Compute & Orchestration
2. Infrastructure as Code
3. CI/CD Pipelines & Deployment
4. Package & Artifact Management
5. Policy & Security
6. Observability
7. Cost & Capacity
8. Self-Service Portal (IDP)

1.5 Key Takeaways

Key Takeaways

1. **DevOps:** Combines software development and IT operations with the goal of continuous delivery of software
2. **The DevOps cycle has 8 stages:** Plan, Code, Build, Test, Release, Deploy, Operate, Monitor - forming a continuous infinity loop between Dev and Ops
3. **CI/CD terminology maps onto the same cycle:** Continuous Integration, Delivery Deployment, and Feedback are just different groupings of those 8 stages
4. **Feedback closes the loop:** Operate and Monitor data flows back into Plan, making the cycle continuous
5. **Platform Engineering builds on DevOps:** It packages DevOps stages into a self-service internal platform so developers don't reinvent CI/CD, IaC, and observability themselves
6. **Internal platforms should be treated as products** - built for developer "customers", not run as a ticket-based service desk.

2. CI/CD with GitLab

Learning Objectives

- Explain CI/CD
- Model a CI/CD pipeline and map it to the DevOps cycle
- Write a functional .gitlab-ci.yml file
- Understand the different testing strategies

CI/CD pipeline

A series of automated steps that helps software teams deliver code faster, safer and more reliably. CI/CD Pipelines are an essential part of continuous integration and deployment and are critical for modern software development.

Benefits

- Faster feedback loops lead to higher quality
- Reduce manual effort and therefore also human errors
- Lead to consistent environments, “works on my machine” disappears
- Parallel jobs allow for scalability in order to process many commits without extra manual effort
- Logged pipeline outputs provide traceability for audits
- Increase efficiency and save costs
- **Enables continuous delivery and deployment**

2.1 Concepts

Concept	Purpose
Stages	Logical grouping of jobs
Jobs	Atomic units of work that run inside a stage
Steps	Individual commands/scripts that a job executes
Rules	Controls when a job runs
Services	Auxiliary containers/services that run alongside a job (databases, Docker-in-Docker, APIs)
Runners	Compute resources that execute jobs (also agents or executors)
Caches	Temporary storage that speeds up repeated work (dependency downloads, compiled objects)
Artifacts	Files produced by a job that are persisted for later stages or for download
Registries	Stores for built images or packages (Docker Hub, GitLab Container Registry)
Variables	Secrets are key-value pairs (plain or masked) injected into jobs for configuration or credentials

2.2 GitLab CI Pipeline Basics

Typical Stages

0 Git commit triggers the pipeline

1 Build

Key Activities

- Compile code
- Create container images
- Run unit, integration, security and performance tests

2 Test

Key Activities

- Run unit, integration, security and performance tests

3 Release

Key Activities

- Publish artifacts to a registry

4 Deploy

Key Activities

- Push the artifact to a target environment (staging, production)

5 Post-Deploy (optional)

Key Activities

- Smoke tests, health checks, notifications, roll-back on failures

Concept	GitLab keyword	Example
Stages	<code>stages</code>	<code>stages: [build, test, deploy]</code>
Jobs	top-level key	<code>job_name: { stage:, script: [...] }</code>
Steps	<code>script</code>	<code>script: ["cmd1", "cmd2"]</code>
Rules	<code>rules</code> / <code>only</code> / <code>except</code>	<code>rules: [{if: ..., when: ...}]</code>
Services	<code>services</code>	<code>services: [{name: "postgres:15", alias: "db"}]</code>

Environments

Describe where code is deployed (Review / Staging / Production / etc.), They can link to a Kubernetes cluster and are declared in the `gitlab-ci.yml` or the GitLab UI.

```
setup-review:
  stage: deploy
  image: alpine/curl:latest
  script:
    - echo "setup review environment for MR ${CI_MERGE_REQUEST_IID}"
  environment:
    name: review/${CI_COMMIT_REF_SLUG}
    url: https://${DEV_DOMAIN_NAME}
    on_stop: stop_review
  rules:
    - if: $CI_MERGE_REQUEST_ID
      when: manual
```

- `environment.name` — dynamically named per merge request using `$CI_COMMIT_REF_SLUG`, so each MR gets its own review environment
- `environment.url` — the link GitLab shows in the MR/pipeline UI to jump straight to that environment
- `environment.on_stop` — references another job (`stop_review`) that tears down the environment when it's no longer needed
- `rules` — this job only runs (manually) when triggered from a merge request

Runners

A worker that executes pipeline jobs that can be run in:

- Local shell
- Via SSH
- Virtualized with Parallels/Virtualbox
- Docker (Autoscaler)
- Kubernetes

A dedicated runner is set up by installing `gitlab-runner` and registering it to the project via a registration token.

Type	Scope	Registered/managed by
Instance runner	Available to all projects/groups on the GitLab instance	GitLab admin
Group runner	Available to all projects within a specific group (and its subgroups)	Group owner/maintainer

Project runner	Dedicated to one specific project (can be shared with select other projects)	Project maintainer
----------------	--	--------------------

A runner is picked by a job via **tags** advertised by each runner, regardless of whether that runner is instance-, group-, or project-scoped.

Key Takeaways

1. Pipeline = stages → jobs → scripts, triggered by git events (push, merge request, tag)
2. Typical stages: Build → Test → Release → Deploy → Post-Deploy (optional)
3. Runners execute the jobs: Can be dedicated to a group/project or instance runners, available to the entire GitLab instance
4. Environments track where code is deployed

3. Infrastructure Provisioning (IaC)

Learning Objectives

- Explain the motivation behind Infrastructure-as-Code (IaC) and its benefits
- Understand and differentiate between imperative and declarative approaches
- Identify the core components of a typical IaC toolchain (language, state, provisioner, backend)
- Write a small IaC script that provisions some simple resources in the public cloud.
- Troubleshoot IaC issues

Infrastructure as Code (IaC)

Managing and provisioning infrastructure through machine-readable definition files, rather than manual configuration.

Benefits

- Enables Automation
- Drift detection
- Repeatability/idempotency - same code produces the same result
- Source of truth via state file
- Team collaboration via remote backends + locking

3.1 Imperative vs. Declarative

	Imperative (procedural)	Declarative (desired-state)
Focus	How to achieve a result	What the result should look like
Control flow	Explicit	Implicit
State/idempotency	Usually stateless, not idempotent	Usually stateful and idempotent

3.2 Terraform/OpenTofu

Terraform

A Open-Source infrastructure as code engine written in GO, created and owned by HashiCorp. It uses the HashiCorp Configuration Language (HCL).

OpenTofu

A Fork of Terraform v1.5.7, under the Linux Foundations stewardship. It is a Drop-in replacement and has feature parity with Terraform. It includes additional features like local state encryption.

HCL & File Conventions

HCL

Human-readable language used to write Terraform files.

- **Declarative:** you describe the final state you want, not the steps to get there
- **Blocks & Arguments:** structure uses { } for blocks, **key** = "value" for settings
- **Logic-Lite:** Supports simple loops, conditionals, and functions (e.g. **count** or **for_each**)
- **Machine-Friendly:** optimized for both human editing and automated API processing
- **File handling:** files end in **.tf**, and Terraform loads every **.tf** file in the working directory

The terraform block is a mandatory bootstrapping block, it tells Terraform which version it needs and which providers (and versions) to fetch during init:

```
terraform {
  required_version = ">= 1.0.0"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 3.68.0"
    }
  }
}
```

File Naming Best Practices

Filename	Purpose
terraform.tf	required_version + required_providers
backend.tf	remote state config (kept separate from versioning)
providers.tf	auth/config for your specific providers
main.tf	primary workspace — resource and data source blocks
variables.tf	input definitions (kept in alphabetical order)
outputs.tf	return values for other modules/CLI (alphabetical order)
locals.tf	derived/computed local values
override.tf	special-use overrides — loaded last; use sparingly, with comments

Terraform Core Concepts

Providers

- **API bridges** that translate Terraform/HCL into specific cloud API calls
- Define the **resource mapping**: which “building blocks” (VMs, databases, networks) are available
- **Multi-Platform**: Connects to AWS, Azure, Google Cloud, SaaS tools, or even local hardware
- **State-aware**: checks if real world infrastructure matches code
- Handle **authentication/credentials** to the backend service
- **You cannot use Terraform without one, every .tf config needs at least one provider block**

```
provider "aws" {
  access_key = var.aws_access_key
  secret_key = var.aws_secret_key
  region     = "us-east-1"
}
```

Resources

- Specific infrastructure objects (VMs, DBs, Networks)
- Declares what you want to create or manage
- **Goal:** Represents the desired final state of a component
- **Example** - Create an EC2 instance in AWS:

```
resource "aws_instance" "web" {
  ami           = "ami-0123456789abcdef0"
  instance_type = "t3.micro"

  tags = {
    Name = "web-server"
  }
}
```

Data

- Fetches information from existing infrastructure
- **External Reference:** uses data not managed by the current code
- Data Lookup:

```
data "aws_ami" "ubuntu" {
  most_recent = true

  filter {
    name     = "name"
    values   = ["ubuntu/images/hvm-ssd/ubuntu-bionic-18.04-amd64-server-*"]
  }

  filter {
    name     = "virtualization-type"
    values   = ["hvm"]
  }

  owners = ["099720109477"] # Canonical
}
```

- Data Usage:

```
# — Look up an AMI by criteria —
data "aws_ami" "ubuntu" {
  # ...
}

# — Defining resources —
resource "aws_instance" "example" {
```

```
ami          = data.aws_ami.ubuntu.id # <- pulled straight from the data source
instance_type = "t2.micro"
# ...
}

# — Output useful ids —
output "ami_id" {
  value       = data.aws_ami.ubuntu.id
  description = "The image ID we interrogated for Ubuntu AMI"
}
```

Variables

- Dynamic values provided at runtime
- Makes configurations flexible without changing code
- Keeps secrets and environment-specific data separate

```
variable "aws_access_key" {
  type       = string
  description = "AWS access key, supply via TF_VAR_aws_access_key"
  sensitive   = true
  # no default -> Terraform prompts interactively if not set
}

variable "instance_type" {
  type       = string
  default    = "t3.micro"
}

# Override variable at runtime:
# terraform apply -var="port=8081"

# Provided as environment variable in the shell:
# export TF_VAR_aws_access_key="AKIA..."
# export TF_VAR_aws_secret_key="..."
```

Outputs

- Data displayed after a successful `apply`
- **Information sharing:** passes values (like an IP address) to the user or to other modules
- **Visibility:** surfaces critical resource attributes directly in the CLI output

```
output "instance_public_ip" {
  value       = aws_instance.web.public_ip
  description = "Public IP of the web server"
}
```

References

- Connects blocks of code together
- **Dependency mapping:** tells Terraform what to build first
- Implicit by default, but can be forced explicit with `depends_on`
- **Attribute access:** pulls a value (like an IP) from one resource into another

- **Consistency:** if a source value changes, everything referencing it updates automatically

```
resource "aws_eip" "web_ip" {
  instance = aws_instance.web.id # implicit reference -> implicit dependency

  depends_on = [aws_instance.web] # explicit dependency (rarely needed; usually automatic)
}
```

Modules

- Containers for grouping multiple resources together
- A reusable, parameterized bundle of Terraform config
- Allows for standardization, enforcing best practices across an organization

```
module "web_server" {
  source      = "./modules/ec2-instance"
  instance_type = var.instance_type
  ami         = "ami-0123456789abcdef0"
}
```

The State File

State

The Source of Truth

- A JSON file mapping your code to real-world resource IDs
- Default stored in `terraform.tfstate`
- It has two Jobs:
 - **Performance** - lets Terraform “remember” what it built previously, so it doesn’t have to rediscover everything from scratch
 - **Drift detection** - catches when someone manually changed a setting in the console

The First time you run `terraform apply`, terraform notices there is no state file, it updates the cloud (creates the resources) and creates the `.tfstate` file as a byproduct

- Flow: `HCL` → `plan` → `apply` → `cloud` → `update .tfstate`

Every subsequent apply is a 3-way comparison:

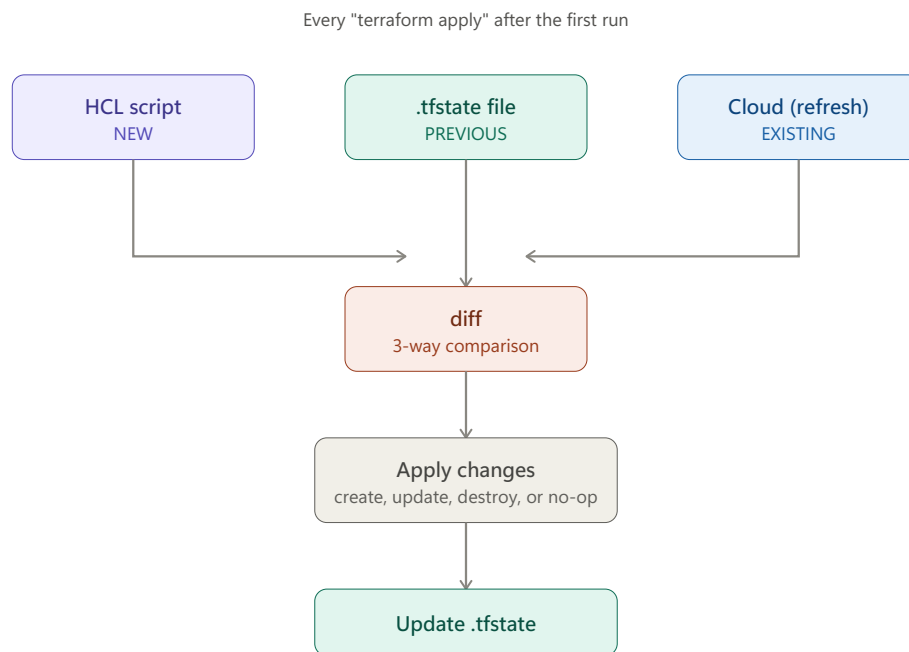


Figure 2: Terraform 3-way Diff

1. NEW — Terraform reads your current script
2. EXISTING — Terraform reaches out to the cloud ([refresh](#)) to see what's actually provisioned right now
3. PREVIOUS — Terraform reads the [.tfstate](#) file from the last run

Terraform reconciles all three to build a full picture of the situation, then decides what action to take (create/update/destroy/no-op per resource)

Example State File:

- The terraform resource is the existing desired state in the script
- The arn is used to check the cloud and see if anything has changed

```
{
  "version": 4,
  "terraform_version": "0.15.1",
  "serial": 4,
  "lineage": "672708eb-8c19-0463-9eb0-81e20b008a9d",
  "outputs": {},
  "resources": [
    {
      "mode": "managed",
      "type": "aws_instance",
      "name": "exercise_0020",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        {
          "schema_version": 1,
          "attributes": {
            "ami": "ami-...",
            "arn": "arn:aws:ec2:us-east-1:641995674308:instance/i-0f5f630af308f2516",
            # truncated for brevity
            "vpc_security_group_ids": [
              "sg-02447ae77b0fda694",
              "sg-02a4ff78bfae29ce2"
            ]
          },
          "dependencies": [
            "aws_security_group.sec_http",
            "aws_security_group.sec_ssh"
          ]
        }
      ]
    }
  ]
}
```

Backends: Team Collaboration

By default, the state file is local. This works fine in a solo environment, to work in teams, the state file should be shared. Terraform supports storing the state in a third party, for example in Amazon S3.

Sequence per `apply`:

1. Acquire lock
2. Fetch `terraform.tfstate` from S3
3. Update the cloud with the desired changes
4. Send the updated `terraform.tfstate` back to S3

Locking rule: if two users run `apply` simultaneously, whoever grabs the lock first wins

Using the Bucket:

Like a forwarding address you put at the top of your project, telling Terraform: “don’t keep your memory (state) on this laptop — keep it over there instead.”

```
terraform {  
  backend "s3" {  
    bucket      = "bucket-name"  
    key         = "backend/terraform.tfstate"  
    region      = "us-east-1"  
    use_lockfile = true  
  }  
}
```

Creating the bucket:

An ordinary resource block — “create an S3 bucket,” then “turn on versioning for that bucket”.

```
resource "aws_s3_bucket" "backend" {  
  bucket = "bucket-name"  
}  
  
resource "aws_s3_bucket_versioning" "backend" {  
  bucket = aws_s3_bucket.backend.id  
  versioning_configuration {  
    status = "Enabled"  
  }  
}
```

⚠ Important: Locking via `dynamodb_table = "tfstate_lock"` is deprecated

Best practices:

- One folder per environment (dev/, prod/, base/, modules/)
- Each folder gets its own state file — never mix state across environments
 - Blast radius containment

Terraform Workflow & Provisioners

Provisioners

Plugins that run scripts on a resource after it’s been created, for post-creation setup. As opposed to providers, which create the resource in the first place.

Common types: remote-exec (runs commands on the remote resource), local-exec (runs commands on the machine running Terraform).

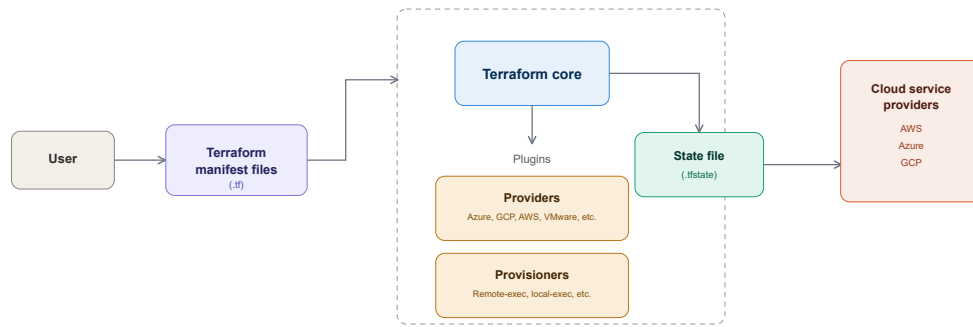


Figure 3: Terraform 3-way Diff

Key Takeaways

1. IaC replaces manual provisioning with code
2. Terraform is declarative, not imperative
3. The toolchain has four named components — language, state, provisioner, backend
4. Providers are the bridge to the real world
5. Resources declare what you want; data sources read what already exists
6. State is the mechanism that makes drift detection possible
7. Backends solve the team-collaboration problem state creates
8. Modules exist for reusability and standardization
9. Workflow: Write HCL → init → plan → apply → destroy - with plan being read-only (preview) and apply being the only step that actually changes anything

4. Configuration Management

Learning Objectives

- Explain the difference between infrastructure provisioning and configuration management (CM)
- Understand the motivation and benefits of configuration management
- Differentiate between imperative and declarative approaches in CM
- Write basic Ansible playbook for server configuration
- Troubleshoot common Ansible issues

Configuration Management

- Comes after infrastructure provisioning in the workflow
- Centrally modifies base configurations
- Rolls out new settings to all applicable systems
- Automates system identification, patching, and updates
- Identifies outdated, poorly performing, and noncompliant configurations
- Example scope: server configuration of packages / users / services

4.1 What is Ansible?

Ansible

An IT Automation tool that:

- Configures systems
- Deploys software
- Orchestrates more advanced IT tasks
 - continuous deployments
 - zero downtime rolling updates

Use cases: Configuration Management, Security & Compliance

Terraform vs. Ansible

	Imperative (procedural)	Declarative (desired-state)
Primary role	Provisioning	Configuration Management
Paradigm	Declarative	Hybrid
Lifecycle	Has state/lifecycle	No Lifecycle
Idempotency	Idempotent	Idempotency optional

4.2 Ansible Architecture & Connection Model

- **Control node:** where Ansible runs (the “management node”)
- **Managed nodes:** target hosts
- Connections happen via
 - **localhost** (local connection)
 - **SSH** (Linux/Unix hosts)
 - **WinRM** (Windows hosts)
- No agent required on managed nodes (agentless)

Inventory

- File listing hosts and variables Ansible will operate on
- Two supported formats: **INI** or **YAML**

Example YAML:

```
--
all:
  hosts:
    r1:
      ansible_host: 192.168.1.10
    r2:
      ansible_host: 192.168.1.11
    sw1:
      ansible_host: 192.168.1.20
    sw2:
      ansible_host: 192.168.1.21
  children:
    router:
      r1:
      r2:
    switches:
      sw1:
      sw2:
  vars:
    ansible_user: cisco
    ansible_ssh_pass: cisco
```

Example INI:

```
[router]
r1 ansible_host=192.168.1.10
[switches]
sw1 ansible_host=192.168.1.20
[all:vars]
ansible_user=cisco
```

Ad-hoc commands (one-off, non-playbook execution):

```
$ ansible [pattern] -m [module] -a "[module options]"
$ ansible webservers -m ansible.builtin.file -a "dest=/srv/foo/a.txt mode=600"
```

- Default module (if -m omitted) is command
- Can target multiple groups: `ansible nodes:webserver -a "shutdown -h now" --become`

4.3 Plays & Playbooks

- **Task:** individual unit of work (e.g. install a package, copy a file)
- **Play:** one or more tasks run in order, defines which hosts to run on. Hosts that fail a task are removed from the rest of the play.
- **Playbook:** a single YAML file containing one or more plays

```
- name: Configure Web Server
  hosts: web_servers
  tasks:
    - name: Install Nginx
      apt:
        name: nginx
        state: present
    - name: Copy website files
      copy:
        src: /local/path/
        dest: /var/www/html/
```

4.4 Core Constructs

Facts

- Automatically collected info about hosts when a playbook runs (“Gathering Facts,” via the `setup` module)
- Stored in `ansible_facts` variable
- Can be disabled with `gather_facts`: no for performance/when not needed

Conditionals

- `when`: clause uses raw Jinja2 expressions
- Can branch on facts, registered variables, or regular variables
- Example pattern: register a command’s exit code, then gate a later task on it:

```
- name: Check if already present
  command: which docker
  failed_when: false
  changed_when: false
  register: docker_available

- name: Download install script
  get_url:
    url: https://get.docker.com/
    dest: /tmp/get-docker.sh
  when: docker_available.rc == 1
```

Loops

- Useful for creating multiple users, installing multiple packages, or repeating a polling step
- Example:

```
- name: add several users
  user:
    name: "{{ item.name }}"
    groups: "{{ item.groups }}"
  loop:
    - { name: 'testuser1', groups: 'wheel' }
    - { name: 'testuser2', groups: 'root' }
```

⚠ **Important:** Some Plugins can take lists as options - better than looping

Handlers

- A task triggered by another task — but only if the triggering task reported a change AND all other tasks succeeded
 - Not run if the triggering task results in “OK” (no change)
- Run after all other tasks complete successfully; skipped entirely if an earlier task fails
- Related flags: `ignore_errors` (continue play despite error), `force_handlers` (run handlers even if some tasks failed)

Ansible Vault (secrets)

- Encrypts sensitive data (passwords, keys) so they aren’t stored in plaintext in playbooks/roles
- Vault files can safely be committed to source control
- CLI commands: `ansible-vault create`, `edit`, `view`, `encrypt_string`
- Password supply methods: file, interactive prompt (`@prompt`), or a secret-manager script (`ansible-vault-keyring-client.py`)
- Usage pattern: reference `vars_files: [vault.yml]` in the playbook, consume encrypted vars via Jinja2 (`{{ username }}`)

4.5 Templating (Jinja2)

- Ansible uses Jinja2 for dynamic expressions/variables
- **Templating happens on the controller**, not the managed node — only the resolved result is sent to the target

Mechanism: the `template:` module takes a `.j2` source file (`src:`) and writes the rendered output to a destination path (`dest:`) on the target.

Example:

playbook.yml:

```
- hosts: localhost
  vars:
    cars: [Porsche, Tesla, Ferrari, Dacia, Citroen, Volvo, Saab]
  tasks:
    - name: template looping example
      template:
        src: template.j2
        dest: thecars.txt
```

template.j2:

```
My Template Cars are
{% for item in cars %}
{{ item }}
{% endfor %}
```

Rendered `thecars.txt`:

```
My Template Cars are
Porsche
Tesla
Ferrari
Dacia
Citroen
Volvo
Saab
```

Filters

- Filters transform data inside a template expression — syntax is `{{ value | filter }}`

```
{{ list | min }}           {# minimum value from a list #}
{{ [3, 4, 2] | max }}      {# maximum value from a list #}
{{ ['a', 'b', 'c'] | shuffle }} {# => ['b', 'c', 'a'] randomize list order #}
{{ ipvar | ipaddr }}       {# test if string is a valid IP #}
{{ ipvar | ipv4 }}
{{ ipvar | ipv6 }}         {# test IP protocol version #}
{{ 'test1' | hash('sha1') }} {# get sha1 hash of a string #}
```

4.6 Tags

- Attribute settable on plays, roles, or tasks
- Lets you run a subset of a playbook instead of the whole thing
- Tags on a play/static import are inherited by included tasks
- Two special reserved tags:
 - `always` — runs unless explicitly skipped with `--skip-tags=always`
 - `never` — does NOT run unless explicitly requested with `--tags=never`

Useful CLI: `ansible-playbook playbook.yml --list-tasks` (shows which tags apply to which tasks)

```
# playbook.yml
---
- hosts: local
  tags:
    - docker
  tasks:
    -name: Install docker
```

```
$ ansible-playbook playbook.yml --list-tasks
playbook: playbook.yml

play #1 (local): local      TAGS: [docker]
  tasks:
    Install docker    TAGS: [docker]
```

4.7 Collections & Roles

Collections

- Modules are being packaged into Collections (e.g. `cisco.ios.ios_banner` instead of bare `ios_banner`)
- Installed via `ansible-galaxy collection install <name>`
- Found on Ansible Galaxy (public, free) or Automation Hub (RedHat-account-gated, certified collections)
- Dependencies declared in a `requirements.yml`, listing both collections and roles, installed in one go

Roles

- A level of abstraction above tasks/playbooks → reusability
- Breaks a complex playbook into smaller, self-contained chunks
- Created via `ansible-galaxy init <name>`, which scaffolds a standard directory structure:

```
cldop/
├── defaults/main.yml
├── files/
├── handlers/main.yml
├── meta/main.yml
├── tasks/main.yml
├── templates/
├── tests/
└── vars/main.yml
```

- Used in a playbook via the `roles:` key instead of `tasks:`

4.8 Best Practices

1. Use roles to group related tasks
2. Use Ansible Galaxy — don't reinvent the wheel; reuse and share roles
3. Don't use `ignore_errors` — it swallows all errors indiscriminately and can leave hosts in a broken/unstable state; use Ansible's built-in error-handling constructs instead (e.g. `failed_when`, `register` + `when`)
4. Use Vault for secrets/sensitive data
5. Use git — centralize and version playbooks for the whole team; even secrets can go in git if Vault-protected
6. Be pragmatic, not clever — reduce complexity, keep it simple:
 - Avoid regex where possible
 - If missing facts, consider adding custom ones
 - Use templates
 - Use tags
 - Use includes/imports rather than one giant 1000-line playbook

Key Takeaways

1. Configuration management happens after infrastructure provisioning
2. Ansible is agentless
3. Templating (Jinja2) runs on the controller, not the target
4. Handlers only fire on change — they're not run if a task reports "OK"
5. Roles and Collections exist for reusability — don't write everything from scratch
6. Secrets belong in Vault, not plaintext — encrypted Vault files can safely live in git
7. `ignore_errors` is a trap, not a fix — it suppresses all failures indiscriminately and risks leaving hosts in a broken state

5. Kubernetes Object Management

Learning Objectives

- Explain what problems Kubernetes helps solving and how
- Understand and explain the purpose of different Kubernetes objects, such as pods, services, deployments, config maps, and secrets
- Understand the difference between imperative commands and declarative configuration for managing objects in Kubernetes and the different usage of the kubectl command
- Understand how to view the history of changes to objects, rollback to previous versions if necessary, and ensure reliable management of object configurations over time

5.1 Foundations

Kubernetes

Kubernetes, also known as K8s, is an open source system for automating deployment, scaling, and management of containerized applications.

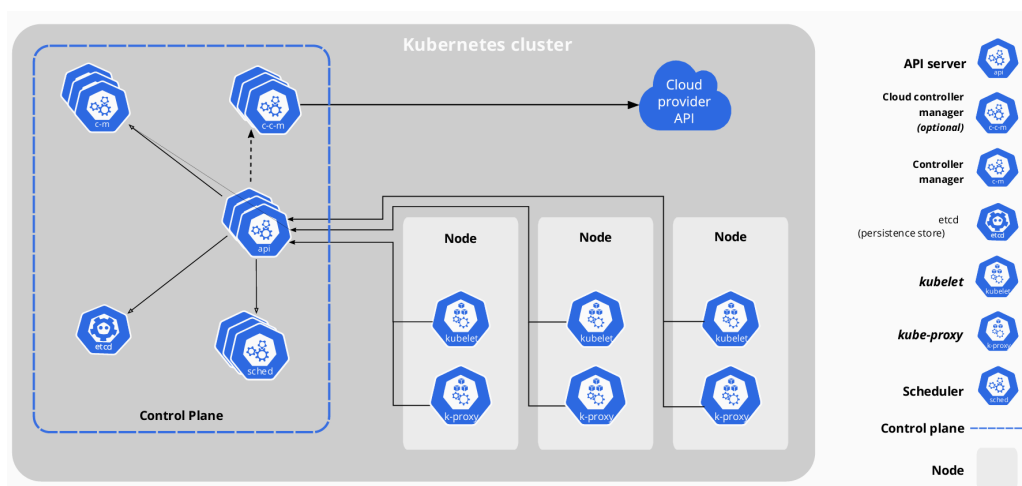


Figure 4: Kubernetes Architecture Overview

Objects

Persistent entities representing a “record of intent”, the desired state of the cluster. Defined declaratively, typically via .yaml manifests.

Controllers

Track a resource type, compare desired vs. current state, and drive the cluster toward the desired state.

5.2 Workloads

Object	Role
Pod	Smallest deployable unit; runs one or more tightly-coupled containers sharing network/storage
Init container	Runs to completion before the main container starts; blocks main container if it fails
Sidecar container	Runs concurrently with the main container, independent lifecycle, shares network/storage namespace, supports probe
ReplicaSet	Ensures a specified number of pod replicas are running, matched via label selectors <code>kubectl scale --replicas=5 rs/app-xy-pod</code> , "Manually scale a ReplicaSet"
Deployment	Higher-level object managing ReplicaSets; provides declarative updates, scaling, and update strategies. Recommended over raw ReplicaSets unless orchestration is needed
HorizontalPodAutoscaler (HPA)	Auto-scales pod replica count based on metrics (CPU/memory via Metrics Server, custom or external); configured with <code>minReplicas</code> / <code>maxReplicas</code> / <code>targetMetric</code> <code>kubectl autoscale deployment php-apache --cpu-percent=50 --min=1 --max=10</code> , "Enable HPA on a Deployment"

5.3 kubectl Usage Patterns

	Imperative	Declarative
Approach	Create/modify resources directly via CLI commands and flags	Write manifest files, apply them with <code>kubectl</code>
Control	Less control	Much more control
Recommendation	Not recommended as primary workflow	Recommended way to work with <code>kubectl</code>

- Create new resources:

```
kubectl create -f nginx.yaml
```

- Push settings from a new manifest:

```
kubectl replace -f nginx.yaml
```

- Apply settings from a manifest:

```
kubectl apply -f nginx.yaml
```

- Generate YAML quickly:

```
kubectl get pods <name> -o yaml --dry-run=client > file.yaml
```

- (then clean up status/fields)
- Inspect a running pod:

```
kubectl exec -it <pod> -- sh
```

5.4 Managing Deployments & Rolling Updates

Rolling Update

The Deployment's task is to ensure enough Pods are always running during an update. The new version is deployed and, once confirmed successful, the old version is taken offline.

Strategy	Behavior	When to use
Recreate	All Pods killed, then new Pods created — causes temporary unavailability	Versions cannot run side by side simultaneously
RollingUpdate	Pods replaced one at a time to guarantee availability — the preferred approach	Default choice; tunable via maxUnavailable / maxSurge

⚠ **Important:** **RollingUpdate** options control rollout behavior precisely:

- **maxUnavailable:** max number of Pods that can be unavailable at the same time during the update
- **maxSurge:** max number of Pods allowed to run **above** the desired replica count during the rollout, to preserve minimum availability

Get details about recent transactions :

```
kubectl rollout history deployment app-xy-deployment
```

Undo a previous change:

```
kubectl rollout undo deployment app-xy-deployment
```

5.5 Namespaces & Resource Management

Namespaces

Logical isolation within one cluster. Allows Resource Organization of Group related objects (pods, services, configs) and Access Control, enforce RBAC policies per namespace.

Default namespaces: **default**, **kube-system**, **kube-public**, **kube-node-lease**

	Kubernetes Namespaces	Linux Namespaces
Level	Orchestration layer (virtual/logical grouping)	Kernel-level primitive

Provides	Logical isolation within a single cluster	Actual OS isolation (PID, network, mount, etc.)
Scope	Namespaced resources (Pods, Services) isolated by default; cluster-wide resources (Nodes, Persistent Volumes) shared	Per-process/container isolation enforced by the kernel

Resource governance

- **Requests** = guaranteed minimum for scheduling
- **Limits** = hard cap (throttling for CPU, eviction for memory)
- **LimitRange** = enforces default min/max per namespace
- **ResourceQuota** = caps aggregate consumption per namespace

⚠ Important: Always define both requests and limits to ensure fair sharing and stability.

5.6 Managing Scheduling

Scheduling

Scheduling makes sure that Pods are matched to Nodes so that kubelet can run them.

- The kube-scheduler determines which nodes will run a Pod
- Different labels can be used to influence where a node will be scheduled
- Other options can also be used to influence Pod scheduling
 - **nodeName** in the Pod spec: verify which node the Pod should run
 - **nodeSelector** in labels: specify how to run a Pod
 - **affinity**, **antiAffinity** or **taints**

Filtering

Eliminates infeasible nodes.

Different options are used to check if a node is eligible to run the Pod:

- **PodFitsHostPorts**: checks if free network ports are available
- **PodFitsResources**: checks if the node has sufficient CPU and Memory resources
- **PodMatchNodeSelector**: checks if the Pod Node Selector matches the Node labels
- **CheckNodeDiskPressure**: checks if a node is reporting a filesystem that is almost full, so that the Pod won't be scheduled there
- **CheckVolumeBinding**: Evaluate if the volumes that are requested can be serviced by the node using bound and unbound PVCs

Scoring

After filtering out nodes, scoring is used to evaluate remaining nodes:

- **SelectorSpreadPriority**: spreads Pods across hosts, considering Pods that belong to the same Service, StatefulSet or ReplicaSet
- **LeastRequestedPriority**: prioritizes nodes with fewer requested resources
- **NodeAffinityPriority**: prioritizes nodes according to node affinity scheduling preferences
- Others

Taints and Tolerations

Taints (on Nodes) repel Pods unless the Pod has a matching **toleration**:

- **NoSchedule** — blocks new Pods
- **PreferNoSchedule** — avoids scheduling unless no alternative
- **NoExecute** — evicts existing Pods too

If a Pod has a toleration, it will ignore the taint

⚠ Important: Taints and tolerations have no effect on DaemonSets

`kubectl cordon` / `kubectl uncordon` — stop new scheduling on a node without evicting existing Pods.

5.7 Networking

Services

Provide stable networking/load balancing for ephemeral Pods via label selectors and a virtual ClusterIP; integrate with readiness probes; support session affinity and traffic subsetting (e.g., blue/green).

Type	Behavior
ClusterIP	Internal-only access within the cluster (default)
NodePort	Exposes service on each node's IP at a static port
LoadBalancer	Exposes externally using cloud provider's load balancer
ExternalName	Maps service to external DNS name

Accessing Pods

Ingress

Manage external access to services (HTTP/HTTPS). A single entry point for multiple services.

Components:

- **Ingress Resource:** Defines routing rules
- **Ingress Controller:** Implements the rules (Nginx, Traefik, etc.)

Features

- Host-based routing
- Path-based routing
- SSL/TLS termination
- Load balancing across backend services

Limitations

- Limited to L7 HTTP/HTTPS traffic
- Custom implementations vary by controller
- Less flexible for advanced traffic management

GatewayAPI (modern replacement for Ingress)

Next-generation API for service networking, more expressive and flexible than Ingress.

Resources:

- **Gateway:** Defines the entry point
- **HTTPRoute:** Specifies how to route HTTP traffic to services
- **TLSRoute:** Specifies how to route HTTPS traffic to services

Advantages

- Standardized across vendors
- Supports multiple protocols (TCP, UDP, gRPC)
- Fine-grained traffic control (weighting, retries, timeouts)
- Better separation of concerns

Best Practices

- Use Gateway API for new projects
- Stick with Ingress for legacy compatibility
- Combine with Service Mesh for advanced needs

5.8 Storage

⚠ **Important:** Container filesystems are ephemeral — die with the container.

Volumes

Used to allocate storage that survives a container and stays available during Pod lifetime. They can directly bind to a specific storage type (NFS)

- At its core, a volume is just a directory, which is accessible to the containers in a Pod
- Kubernetes supports several types of volume backends
- A **Persistent Volume** defines access to external storage available in a specific cluster
- **Persistent Volume Claims** (PVC), decouple a Pod from a specific PV. It searches for a matching PV, if none exists, a **StorageClass** dynamically provisions one

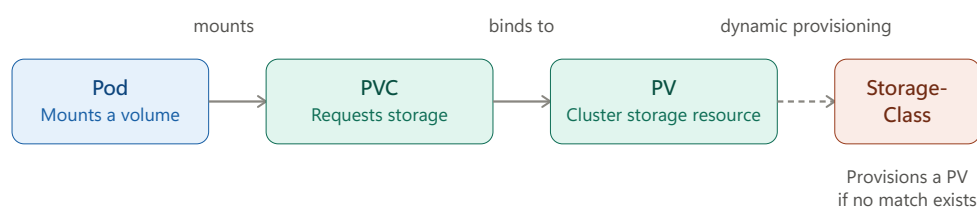


Figure 5: Kubernetes PVC-PV Flow

5.9 ConfigMaps and Secrets

	ConfigMap	Secret
Purpose	Non-sensitive key-value config	Sensitive data (passwords, tokens, keys)
Encoding	Plain	Base64-encoded by default (not encrypted at rest by default)
Scope	Namespace-scoped	Namespace-scoped

Usage	Env vars, mounted files, CLI args	Env vars, mounted files, private image pulls
Types	-	Opaque, Docker registry creds, TLS certs, service account tokens
Best practice	Version-control alongside manifests	• Enable etcd encryption at rest • Use External secret managers for production • Never commit to version control • Limit Access with RBAC policies

⚠ Important: Secrets are not truly secret without encryption at rest

5.10 Probes

Probes

Used to test access to Pods and are part of the Pod specification. The probe itself is a simple test, which is often a command.

Probe types defined in `pod.spec.containers`:

- **exec**: a command is executed and returns a zero exit value
- **httpGet**: an HTTP request returns a response code between 200 and 399
- **tcpSocket**: connectivity to a TCP socket (available port) is successful

Probe	Purpose
readinessProbe	Gates whether a Pod is marked available for traffic
livenessProbe	Continuously checks Pod health, can trigger restarts
startupProbe	For legacy apps needing longer init time (rarely used)

Key Takeaways

1. Kubernetes objects are a record of intent — controllers continuously work to bring current state in line with desired state
2. Always prefer Deployments over bare ReplicaSets unless you need custom update orchestration
3. Declarative `kubectl apply` is the recommended way to manage objects — it gives more control than imperative CLI commands
4. RollingUpdate is the preferred update strategy over Recreate, since it avoids downtime by replacing Pods incrementally
5. `maxUnavailable` and `maxSurge` are the two levers that tune how aggressive or cautious a rolling update is
6. Kubernetes namespaces are a logical/orchestration-layer construct, distinct from Linux kernel namespaces
7. Resource requests guarantee scheduling minimums, while limits cap usage and trigger throttling (CPU) or eviction (memory)
8. The scheduler places Pods in two phases — filtering out infeasible nodes, then scoring the rest to pick the best fit
9. Taints repel Pods unless a matching toleration is present
10. Services give ephemeral Pods a stable network identity via label selectors and a virtual ClusterIP

11. Gateway API is the modern successor to Ingress, but Ingress remains relevant for legacy compatibility
12. Volumes solve the problem of ephemeral container storage; PVCs decouple Pods from specific storage, and StorageClasses provision dynamically when no PV matches
13. Secrets are base64-encoded, not encrypted, by default — they require explicit etcd encryption at rest to be genuinely secret
14. Readiness, liveness, and startup probes each answer a different question about Pod health, using exec, httpGet, or tcpSocket checks

6. Kubernetes Resource Packaging

Learning Objectives

- Differentiate between Helm (package manager) and Kustomize (configuration overlay tool)
- Explain the concept of "Chart" and "Release" in Helm
- Understand the "Base and Overlay" pattern in Kustomize
- Demonstrate how to package an application for multiple environments
- Critique the trade-offs between Helm and Kustomize

6.1 Helm

Helm

Helm is a package manager for Kubernetes that simplifies application deployment and management. Instead of manually creating and maintaining dozens of separate configuration files, Helm bundles everything needed for an application into a single, reusable package called a Helm chart.

Core Concepts

Term	Meaning
Chart	Collection of files describing related K8s resources (like a .deb/.rpm)
Release	Running instance of a chart (e.g., my-app-v1)
Repository	Place where charts are stored/versioned
Template Engine	Go templates used to inject variables into YAML
Values	Externalized configuration passed into templates

Release states (via helm status): `deployed` · `failed` · `superseded` · `uninstalling` · `uninstalled` · `pending-install` · `pending-upgrade` · `pending-rollback` · `unknown`

Helm Charts

A chart is a collection of files describing related K8s resources

```
mychart/
├── Chart.yaml           ← chart description (metadata)
├── values.yaml          ← default values used in chart
├── charts/              ← dependency charts go here
├── templates/
│   ├── _helpers.tpl     ← helper functions (not rendered directly)
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── hpa.yaml
│   ├── serviceaccount.yaml
│   ├── NOTES.txt        ← printed to the user after install
│   └── tests/
└── .helmignore
```

Required fields in a `Chart.yaml`:

Field	Meaning
apiVersion	Chart API version — v2 for Helm 3 / Helm 3 features, v1 for older or backwards-compatible charts
name	Name of the chart
version	SemVer 2 version of the chart itself; must match the version in the package filename

⚠ Important: Every chart must have a version, and it must follow MAJOR.MINOR.PATCH:

- MAJOR — incompatible API changes
- MINOR — backwards-compatible new functionality
- PATCH — backwards-compatible bug fixes
- Pre-release/build metadata labels are allowed as extensions to that format

Dependencies:

- Stored physically in the chart's `charts/` directory
- Helm v3: declared in a `dependencies:` block inside `Chart.yaml` itself
- Each dependency entry has:

Field	Meaning
name	Must match the name in the dependency chart's own <code>Chart.yaml</code>
version	Semantic version or a version range
repository	URL or local path to where that chart lives

```
# remote
dependencies:
- name: nginx
  version: "1.1.1"
  repository: "https://repo.com/charts"

# local
dependencies:
- name: nginx
  version: "1.1.1"
  repository: "file:///../other-charts/nginx"
```

```
helm dependency build    # rebuild charts/ from the lock file
helm dependency list     # show declared dependencies
helm dependency update   # download deps (from Chart.yaml or requirements.yaml) into charts/ository:
                        "file:///../other-charts/nginx"
```

Templates

A Helm template is a YAML file with placeholders. Helm fills in the placeholders using values, then hands kubectl a normal manifest. It's built on Go's text/template engine — not a YAML-native templating system. It supports general-purpose logic (conditionals, loops, variables)

```
{{ .Values.replicaCount }} → Helm fills this in → replicas: 3
```

Objects

Object	What it gives you
--------	-------------------

.Values	Values from values.yaml or other specific files - empty by default
.Release	Describes the release itself
.Chart	Everything accessible from Chart.yaml - example: {{ Chart.Name }}

Functions

```
{{ .Values.name | upper | quote }}
```

Take the value → uppercase it → put quotes around it

Logic

- **if/else** → include something only if a condition is true
- **range** → loop over a list
- **with** → temporarily say “now . means this specific value” so you don’t have to keep retyping the full path

Helm lifecycle commands

```
helm create <name>          # scaffold a chart
helm install <release> <chart> # deploy
helm upgrade <release> <chart> --set ... # update
helm rollback <release> <revision> # revert (use helm history to find revision #)
helm uninstall <release>        # remove resources + history
helm list                      # show releases
helm status <release>          # deployment time, namespace, state, resources
helm history <release>         # revision list
helm search repo               # find charts
helm template <chart>          # render locally without installing
helm get manifest <release>    # download rendered YAML
helm install --dry-run --debug # simulate install, verbose
```

1 Install

Key Activities

- `helm install <release> <chart>` deploys the chart as a new release
- Optionally test first: `--dry-run --debug` simulates without applying

Tools

`helm install` · `helm template`

2 Upgrade

Key Activities

- `helm upgrade <release> <chart>` updates an existing release
- `--set` overrides specific chart values at upgrade time

Tools

`helm upgrade`

3 Rollback

Key Activities

- `helm history <release>` lists past revision numbers
- `helm rollback <release> <revision>` reverts to a prior revision

Tools

`helm history` · `helm rollback`

4 Uninstall

Key Activities

- `helm uninstall <release>` removes all resources from the last release
- Release history is also removed by default

Tools

`helm uninstall`

6.2 Kustomize

Kustomize

Kustomize is a tool for customizing Kubernetes configurations. It takes plain K8s YAML you already have and layers changes on top of it declaratively, via `kustomization.yaml`

- **Base** = what's common across all variants
- **Overlay** = the differences for one specific variant (e.g. dev, prod)
- Output is a “variant” — pure rendered YAML
- Build per environment: `kustomize build overlays/prod`

Example directory layout:

```
~/someApp
├── base/
│   ├── deployment.yaml
│   ├── kustomization.yaml
│   └── service.yaml
└── overlays/
    ├── development/
    │   ├── cpu_count.yaml
    │   ├── kustomization.yaml
    │   └── replica_count.yaml
    └── production/
        ├── cpu_count.yaml
        ├── kustomization.yaml
        └── replica_count.yaml
```

Kustomize commands:

```
kustomize create --resources <file> # scaffold kustomization.yaml
kustomize edit                      # modify kustomization.yaml programmatically
                                   # (add resource, set image, add label, remove resource)
kustomize build <dir>              # render final manifest (often piped to kubectl apply -f -)
kustomize cfg fmt <dir>            # format for consistency
kustomize localize <dir>           # bundle remote (URL/Git) resources locally for offline use
```

Transformers

Built-in functions that “transform” your manifests using simple declarative rules. You declare what you want changed in kustomization.yaml; Kustomize applies it.

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
metadata:
  name: demo

resources:
- base/nginx-app.yaml

labels:
- includeSelectors: true
  pairs:
    app.kubernetes.io/name: hello

namespace: cldop

commonAnnotations:
  version: 1.0.0

namePrefix: kustom-
nameSuffix: -v1
```

You can: add labels, set a namespace, add annotations, and prefix/suffix every resource name — all without touching the original manifests

Patches

A patch is a partial spec — it doesn’t redefine a whole resource, just describes what one section should look like after the change.

- It can either **replace** an existing value or **append** to a list; which one happens depends on the patch operation (op:) used
- Kustomize needs enough info (target: kind/name) to know exactly which resource/field you’re patching, otherwise it can’t apply it correctly.

Example — renaming a Service via patch:

```
patches:
- patch: |-
  - op: replace
    path: /metadata/name
    value: nginx-server
  target:
    kind: Service
    name: nginx-app
```

Result: nginx-app → kustom-nginx-server-v1 (the patch rename plus the namePrefix/nameSuffix transformers from the kustomization file both apply).

ConfigMaps/Secrets in Kustomize

	Option 1: Traditional Model	Option 2: Generator Mode
Definition	Plain ConfigMap/Secret YAML, included via resources:	configMapGenerator / secretGenerator in kustomization.yaml
Behavior on change	Pods' mounted config updates without a rolling update	kustomize build appends a content hash to the name, so referencing Deployments see a changed name → triggers rolling update

Secret generator can build from **literals**: (key=value pairs) or **files**: (e.g. a password.txt), both rendered into a base64-encoded **Secret** manifest with a hashed name suffix.

6.3 Comparison & Best Practices

Requirement	Recommended Tool
Distribute a complex app (DB + App)	Helm
Manage env-specific configs (Dev/Prod)	Kustomize
Avoid learning a new templating language	Kustomize
Consume community charts	Helm
Need loops/conditionals	Helm

Best practices

- Keep custom resources and their instances in separate packages
- Keep common values (namespace, metadata) in the base
- Organize resources by kind (e.g. horizontal-pod-autoscaler.yaml, services.yaml)
- Follow standard directory structure: bases, patches, overlays

Common pitfalls

- **Helm**: over-complicating templates — keep them simple
- **Kustomize**: too many nested overlays — flatten the structure
- **Both**: not pinning image versions
- **Both**: ignoring state management (Helm release drift / Kustomize drift)
- **Both**: not validating manifests before applying

Key Takeaways

1. Helm is a package manager that templates and versions releases; Kustomize is an overlay tool that patches plain YAML with no templating language.
2. A Chart is the packaged unit; a Release is a running instance of that chart in the cluster, and one chart can produce multiple releases.
3. Kustomize's Base/Overlay pattern keeps common config in the base and environment-specific differences in overlays.

4. Use Helm for complex multi-component apps, community charts, or when you need loops/conditionals; use Kustomize for environment-specific configs or to avoid learning a templating language.
5. Editing a traditional ConfigMap doesn't trigger a rolling update, but Kustomize's configMapGenerator/secretGenerator hashes the name on change, which does trigger one.
6. Common pitfalls include over-complicated Helm templates, deeply nested Kustomize overlays, unpinned image versions, ignored drift, and skipping manifest validation before applying.
7. The Helm lifecycle follows install → upgrade → rollback (via helm history) → uninstall, with helm status reporting release state independently at any point.

7. Kubernetes Security — Cilium Networking and Policies

Learning Objectives

- eBPF Basics
- Cilium Networking
- Cilium Network Policy Deep Dive
- Getting Started with Cilium Network Policies

eBPF

A technology that runs sandboxed programs in the Linux kernel without changing kernel source code, used by Cilium for networking/security.

Concept	Role
BPF programs	Code logic that runs at kernel hook points
BPF maps	Persist state between program invocations (datapath config, observability counters, etc.)

Cilium

An eBPF-powered Container Network Interface (CNI) for Kubernetes that provides high-performance networking, deep observability, and security enforcement

7.1 Cilium Networking

Kubernetes Network Policy

A declarative, pod-level network firewall abstraction that understands Kubernetes identities (like labels) rather than just IP addresses. The standard, built-in Kubernetes resource for basic L3/L4 policies

CiliumNetworkPolicy

A Custom Resource Definition (CRD) that is a superset of the standard policy, adding advanced features like L7 (API-aware) rules and FQDN filtering

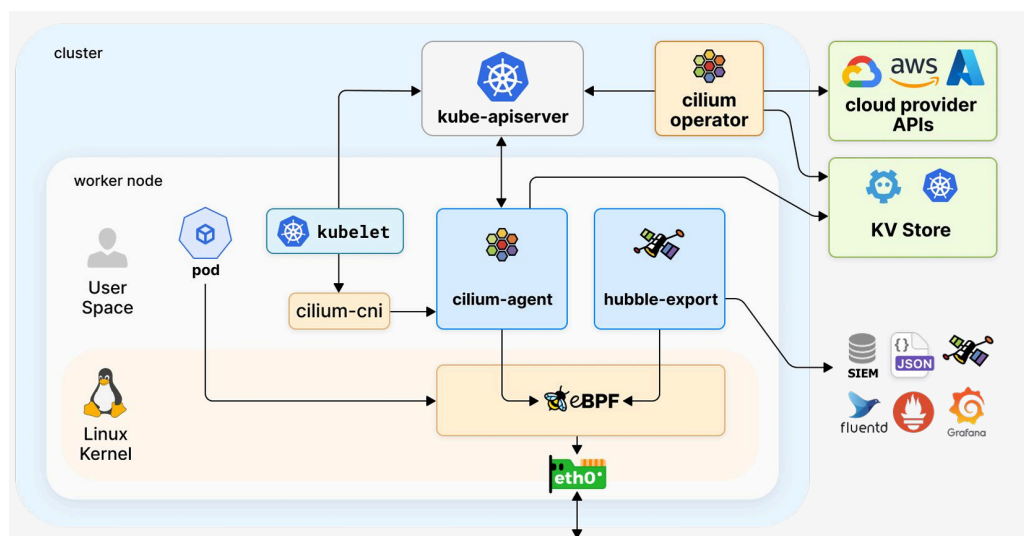


Figure 6: Cilium Architecture - High-level View

Architecture components:

Concept	Role
cilium-operator	Talks to kube-apiserver and cloud provider APIs / KV store
cilium-cni	CNI plugin invoked by kubelet
cilium-agent	Per-node daemon, programs eBPF
hubble-export	Exports flow data to SIEM/JSON/Grafana/etc.

Cilium does not enforce policy on IP addresses. Pods get a security identity derived from their labels (e.g., role=backend → identity 20). Policy decisions are identity-to-identity, which is why churning pod IPs don't break policies.

Feature-by-feature comparison between NetworkPolicy and CiliumNetworkPolicy:

Aspect	NetworkPolicy	CiliumNetworkPolicy
Origin	Built-in Kubernetes API object	Cilium Custom Resource Definition (CRD)
API kind	NetworkPolicy	CiliumNetworkPolicy (CNP)
Layer support	L3 / L4 only	L3 / L4 / L7 (HTTP, gRPC, Kafka, DNS, ...)
Selectors	podSelector	endpointSelector, incl. matchExpressions
CIDR support	ipBlock (basic)	toCIDR / fromCIDR, toCIDRSet / fromCIDRSet
FQDN / DNS-aware rules	Not supported	toFQDNs (requires a DNS proxy rule)
Reserved entities	Not supported	toEntities/fromEntities: world, cluster, host, remote-node, kube-apiserver
Cluster-wide scope	Not supported (namespaced only)	CiliumClusterwideNetworkPolicy variant available
Port ranges	Not supported	endPort keyword (Cilium 1.16+)
Relationship	Subset	Superset — includes everything NetworkPolicy can express, plus advanced Cilium features

7.2 Cilium Network Core Concepts

Declarative Policy

- Allowed connectivity is defined using declarative Kubernetes resources
- **NetworkPolicy**: built-in K8s abstraction for basic policies
- **CiliumNetworkPolicy**: Custom Resource Definition (CRD) that is a super-set of NetworkPolicy, includes advanced Cilium features
- Each network policy resource defines one or more “**rules**”

Endpoint Selection

- Each policy exists in a namespace, except for cluster-wide network policies
- Endpoint selectors match pods within that namespace, or cluster-wide
- A policy matches 0 or more pods in a namespace based on a **label-match**:
 - podSelector field in NetworkPolicy
 - endpointSelector field in CiliumNetworkPolicy
- **Subset Match**: pod must have the labels specified in matchLabels, but may have other labels as well
- Each rule is applied to all pods that match

Ingress & Egress Rules

- **Ingress Rules:** traffic entering a pod.
- **Egress Rules:** traffic leaving a pod.
- Policies may have any combination of ingress & egress rules
- Enforcement is split by direction and side of the connection:
 - **Pod A Egress Enforcement** — connectivity leaving Pod A
 - **Pod B Ingress Enforcement** — connectivity entering Pod B
 - Same logic applies at the cluster boundary: connectivity **entering the cluster** is enforced at the destination pod's ingress; connectivity **leaving the cluster** is enforced at the source pod's egress.

Default Deny Model

- Each rule describes an **allowed** type of connectivity
- Total allowed connectivity is the **logical OR** of all rules applied to a pod
- Connectivity not explicitly allowed is **implicitly denied**

Automatically Allowed Connections

- Default deny model does **NOT** apply:
 - On ingress for a pod if no ingress rules matches the pod.
 - On egress for a pod if no egress rules matches the pod.
- **All ingress connectivity from K8s worker nodes is automatically allowed**
 - Avoid unintentional impact on network liveness/readiness probes from kubelet.
 - Allows connectivity from host-network pods. Host-network == trusted.

Stateful Connection Tracking

- **Connection reply traffic is automatically allowed**
- Mechanism:
 - egress enforcement on the initiating side does a conntrack match, the connection is initiated (e.g. TCP: 10.0.0.1:12345 → 1.2.3.4:80)
 - ingress enforcement on the receiving side does its own conntrack match for the connection reply (TCP: 1.2.3.4:80 → 10.0.0.1:12345) and allows it automatically.

Policy Layers

L3: Network Endpoints:

- toEndpoints / fromEndpoints
- toCIDR / fromCIDR
- fromCIDRSet / toCIDRSet
- toFQDN

L4:

- toPort / fromPort

L7:

- DNS
- HTTP, gRPC

- Kafka, Memcached, Cassandra

7.3 Policy Definition

Policy Layers

- Within a rule, each layer is logically AND'd with the other layers
- Unspecified layers are wildcards

Pattern	Meaning
L3-only	matches from frontend, all L4 ports allowed
L4-only	matches all L3 remote endpoints, only port 80/TCP allowed
L3 + L4	matches from frontend AND port 80/TCP — both must hold

```
# L3-only (all L4 ports)
ingress:
- fromEndpoints:
  - matchLabels:
    role: frontend

# L4-only (all L3 remote endpoints)
ingress:
- toPorts:
  - ports:
    - port: "80"
    protocol: TCP

# L3 + L4
ingress:
- fromEndpoints:
  - matchLabels:
    role: frontend
toPorts:
- ports:
  - port: "80"
  protocol: TCP
```

Multiple Rules per Policy

- 2 separate rule list items → logical OR
- 1 combined rule (fields nested under the same -) → logical AND

```
# 2 Rules (logical OR)
ingress:
- fromEndpoints:
  - matchLabels:
    role: frontend
- toPorts:
  - ports:
    - port: "80"
    protocol: TCP

# 1 Rule (logical AND)
ingress:
- fromEndpoints:
  - matchLabels:
    role: frontend
```

```
toPorts:
- ports:
  - port: "80"
    protocol: TCP
```

Advanced Endpoint Selectors

Selector type	Behavior
Multi-label Match	Pod must have all labels to match (logical AND) — e.g. <code>org: empire</code> AND <code>class: deathstar</code>
Wildcards	<code>endpointSelector: {}</code> matches all endpoints
Set-Based Label Selectors	More complex/powerful matching via <code>matchExpressions</code> (e.g. <code>key: k8s-app</code> , <code>operator: NotIn</code> , <code>values: [app1, app2]</code>)

```
# Multi-label Match (AND)
spec:
  endpointSelector:
    matchLabels:
      org: empire
      class: deathstar

# Wildcards
spec:
  endpointSelector: {}

# Set-Based Label Selectors
spec:
  endpointSelector:
    matchExpressions:
      - key: k8s-app
        operator: NotIn
        values:
          - app1
          - app2
```

toEndpoints/fromEndpoints

- **Subset match:** pod must have all labels specified in `matchLabels` to be allowed by the rule, but may have additional labels as well
- Namespaces are the key consideration when matching on labels

Binding type	How it works
Implicit Local Namespace Binding	Plain <code>matchLabels</code> — implicitly scoped to the same namespace as the policy
Explicit Namespace Binding	Add <code>k8s:io.kubernetes.pod.namespace:</code> to <code>matchLabels</code> to target a specific other namespace
Match all endpoints in local namespace	<code>matchLabels: {}</code> — wildcard within local namespace
Wildcarding Namespace	Use <code>matchExpressions</code> with <code>key: /</code> <code>operator:</code> to match across any namespace

```
# Implicit Local Namespace Binding
ingress:
- fromEndpoints:
  - matchLabels:
    role: frontend
```

```
# Explicit Namespace Binding
ingress:
- fromEndpoints:
  - matchLabels:
    role: frontend
    k8s:io.kubernetes.pod.namespace: foo

# Match all endpoints in local namespace
ingress:
- fromEndpoints:
  - matchLabels: {}

# Wildcarding Namespace
ingress:
- fromEndpoints:
  - matchExpressions:
    - key: k8s:io.kubernetes.pod.namespace
      operator: Exists
    - key: app
      operator: In
      values:
      - backend
```

toCIDR/fromCIDR and toCIDRSet/fromCIDRSet

```
# toCIDR
egress:
- toCIDR:
  - 10.0.0.0/8

# toCIDRSet (with exception)
egress:
- toCIDRSet:
  - cidr: 0.0.0.0/0
    except:
    - 169.254.169.254/32
```

⚠ Important: CIDR and CIDR-based policies apply ONLY to IPs that are outside the cluster. Do not use them to try and match on Pod, Service, or Worker Node IPs.

toFQDNs L3 Policies

Mode	Example
Exact match	matchName: "www.domain.net" — automatically translated from DNS names, similar to toCIDR
Pattern match	matchPattern: "*.company.com" — regex-based matching

```
egress:
- toFQDNs:
  - matchName: "www.domain.net"

egress:
- toFQDNs:
  - matchPattern: "*.company.com"
```

⚠ Important: Try to avoid using overlapping toFQDNs — especially across multiple policies.

toFQDNs DNS proxy requirement

- toFQDNs rules can only be applied to pods that also have a DNS proxy rule that intercepts all DNS lookups from that pod
- For security reasons, this rule should be limited to DNS servers that you trust (cluster-internal, e.g. only to core-dns).

```
toEndpoints:
- matchLabels:
  io.kubernetes.pod.namespace: kube-system
  k8s-app: kube-dns
toPorts:
- ports:
  - port: "53"
    protocol: ANY
rules:
  dns:
  - matchPattern: "*"

```

toEntities/fromEntities + Reserved L3 Identities

Special reserved identities that correspond to a group of endpoints or notable entities:

Entity	Meaning
world	all endpoints outside the cluster
cluster	all managed pod endpoints
host	the local K8s worker node
remote-node	non-local K8s worker nodes
kube-apiserver	remote nodes which have healthy backends serving the kube-apiserver

```

ingress:
- fromEntities:
  - world

egress:
- toEntities:
  - cluster
  - host
  - remote-node

```

toPorts

Pattern	Behavior
L4-only	implicitly allows from all L3 entities
L3 + L4	allows only connections that match at both layers

```

# L4-only – implicitly allow from all L3 entities
ingress:
- toPorts:
  - ports:
    - port: "80"
      protocol: ANY

# L3 + L4 – match at both layers
egress:
- toPorts:
  - ports:
    - port: "80"
      protocol: ANY
  fromEndpoints:
  - matchLabels: {}

```

Port Ranges

- **With Cilium 1.16+:** implementation of the `endPort` keyword, allowing port ranges in `CiliumNetworkPolicies` and `CiliumClusterwideNetworkPolicies`.

```

ingress:
- fromEndpoints:
  - matchLabels:
    app: client
  toPorts:
  - ports:
    - port: "8080"
      endPort: 8082
      protocol: TCP

```

L7 Policy Example (HTTP)

Same L4-only vs. L3+L4 pattern, one layer up — now with an http rule under toPorts:

Pattern	Behavior
L4 + L7	implicitly allows from all L3 entities
L3 + L4 + L7	allows only connections that match at all layers

```
# L4 + L7 – implicitly allow from all L3 entities
```

```
ingress:
```

```
- toPorts:
```

```
  - ports:
```

```
    - port: "80"
```

```
      protocol: TCP
```

```
    rules:
```

```
      http:
```

```
        - method: GET
```

```
          path: "/path"
```

```
# L3 + L4 + L7 – match at all layers
```

```
egress:
```

```
- fromEndpoints:
```

```
  - matchLabels:
```

```
    app: frontend
```

```
toPorts:
```

```
- ports:
```

```
  - port: "80"
```

```
    protocol: TCP
```

```
  rules:
```

```
    http:
```

```
      - method: GET
```

```
        path: "/path"
```

7.4 Common patterns

Default Deny Policy

- Select all endpoints in namespace, specifying only an “empty” rule for ingress + egress
- Subverts the “automatic allow” by ensuring that all endpoints have at least one ingress + egress rule, but doesn’t permit any traffic.
- Can be installed by default in a namespace, requiring the app team to add explicit “allow” exceptions.

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: deny-all
  namespace: my-namespace-xy
spec:
  description: Block everything
  endpointSelector: {}
  ingress:
  - {}
  egress:
  - {}
```

⚠ Important: This example will even deny namespace internal and traffic toward core-dns/Kubernetes API!

Allow within Namespace:

A variant of the pattern above: instead of an empty rule (which permits nothing), use {} as the match value inside fromEndpoints/toEndpoints — this allows all in-namespace traffic while still denying everything else

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: allow-within-namespace
  namespace: my-namespace-xy
spec:
  description: Allow NS internal traffic, block everything else
  endpointSelector: {}
  ingress:
  - fromEndpoints:
    - {}
  egress:
  - toEndpoints:
    - {}
```

⚠ Important: This example will deny traffic toward core-dns/Kubernetes API!

Default DNS Proxy Rule:

- Before applying a “default deny,” it is recommended to apply a DNS proxy rule — per namespace, or as a CiliumClusterwideNetworkPolicy — to allow and observe all DNS lookups.
- This is the standard fix for the “Careful” warnings on slides 43–44: add this rule first, then layer the deny-all / allow-within-namespace pattern on top.

```
- toEndpoints:
  - matchLabels:
      io.kubernetes.pod.namespace: kube-system
      k8s-app: kube-dns
  toPorts:
    - ports:
        - port: "53"
          protocol: ANY
      rules:
        dns:
          - matchPattern: "*"
```

L7 Visibility Policies

- Provides visibility into API-layer requests, even if no policy is being enforced
- For L7 policy, an “empty rule” matches any L7 request, resulting in wildcard behavior
- Visibility policies can be ingress, egress, or both
- Must remember to also have a rule that allows all traffic on other ports as well — the visibility rule only covers the port it’s attached to

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: http-ingress-visibility
spec:
  endpointSelector:
    matchLabels: {}
  ingress:
    - fromEntities:
        - all
      toPorts:
        - ports:
            - port: "80"
              protocol: "TCP"
          rules:
            http:
              - {}
```

7.5 Getting Started with Cilium Network Policies & Hubble

Hubble

Cilium's observability layer, built on the same eBPF data the dataplane already collects. Three interfaces onto that data:

- Hubble UI — service dependency maps, flow filtering, policy viewer
- Hubble CLI — detailed flow visibility, filtering, JSON output
- Hubble Metrics — feeds Grafana/Prometheus for ops & application monitoring

The recommended workflow:

1. Handle network policies as early as possible — not as an afterthought.
2. Build them bottom-up: nodes (optional) → Kubernetes → Cilium → infrastructure components → user workloads.
3. Continuously watch for DROPPED connections via Hubble.
4. Iterate until no false positives remain

Common Gotchas

- Enabling egress enforcement rules, but not also allowing DNS connectivity
- Creating toFQDNs rules without corresponding DNS proxy rule
- Creating an FQDN policy for internal k8s services
- Creating rules based on the label-identity of unmanaged pods.
- Creating rules to allow connections to pods based on IPs/CIDRs.
 - Use toEndpoints instead
- Creating rules to allow connections to K8s worker nodes based on IPs.
 - Use toEntities with host and remote-node instead

Key Takeaways

1. eBPF makes the kernel safely programmable
2. Cilium enforces policy on identity, not IP
3. CiliumNetworkPolicy is a strict superset of NetworkPolicy, but adds L7, FQDN, CIDR sets, reserved entities, and cluster-wide scope
4. Default-deny has two important exceptions: it only applies once some rule exists for that direction and traffic from worker nodes / host-network pods is always auto-allowed regardless of policy
5. Within a rule, layers are AND'd; across rules, they're OR'd
6. Connection replies are automatically allowed via conntrack
7. CIDR rules only apply outside the cluster
8. toFQDNs needs a DNS proxy rule to function at all
9. The practical workflow is observe-then-restrict: use Hubble to see real traffic before tightening policy

8. Service Mesh

Learning Objectives

- No dedicated "Learning Objectives"

Microservices

A style of application architecture where a collection of independent services communicate through lightweight APIs, taking a cloud-native approach so each core function can exist independently.

- **Pros:** independent deployment, flexibility, scalability, fault isolation, team autonomy
- **Cons:** operational complexity, distributed system challenges, data consistency, possibly higher resource usage, increased testing overhead

Microservices introduce challenges: load balancing/traffic mirroring/rate limiting, fault tolerance, secure communication, observability/debugging. That's where a service mesh comes to play.

Service Mesh

A service mesh is a dedicated infrastructure layer within a software application that handles communication between services, handling traffic routing, security, observability, and resiliency while abstracting that complexity away from individual services.

Capabilities:

- **Traffic Management** (fine-grained routing, traffic splitting)
- **Resilience** (fault tolerance, automatic retries/fallback)
- **Security** (encrypted comms, authN, authZ)
- **Observability** (metrics, logs, tracing)

8.1 Core components

Originally, every app had to implement these features itself. The fix: embed the mesh logic in a sidecar proxy running alongside (not inside) the app — transparently intercepting traffic, no app code changes needed.

- **Data plane:** lightweight proxies (e.g. Envoy) handling communication, metrics, security enforcement
- **Control plane:** manages config/policy (e.g. Istiod) and pushes rules to the data plane dynamically

Sidecars have two major downsides:

- Invasiveness
 - modifies pod spec
 - redirects in-pod traffic
 - requires pod restarts to upgrade
- Underutilization of resources
 - every workload needs its own proxy provisioned for worst-case load

8.2 Sidecarless Service Mesh

L2–L4 (Ethernet/IP/TCP-UDP) can move into the kernel via eBPF, but L7 (HTTP, gRPC) parsing still requires a userspace proxy. The general approach is to split service mesh functionality into two tiers.

Tier	Functions	Cost
Secure Overlay Layer	TCP routing, mTLS tunneling, simple authZ, TCP metrics/logging	streamlined, low resource, zero-trust
TL7 Processing Layer	HTTP routing/LB, circuit breaking, rate limiting, fault injection, retries/timeouts, rich authZ, HTTP metrics/tracing	everything above plus this — i.e. strictly more expensive

Cilium VS Istio Approach

	Istio	Cilium
L3/L4 layer	ztunnel (userspace daemon, 1/node)	eBPF programs (in-kernel, no daemon)
L7 layer	Waypoint proxy (Envoy, 1/namespace, opt-in)	Per-node Envoy (opt-in per flow)
L4 traffic touches userspace?	Yes (ztunnel is still a userspace process)	No — stays in-kernel
Node-to-node L4 transport	Node-to-node L4 transport	Native eBPF-accelerated path

Istio: Split is two distinct components (ztunnel, Waypoint proxy), with two distinct deployment scopes (node vs. namespace), each handling one layer.

ztunnel handles mTLS, L4 telemetry, authentication, authorization Node-to-node traffic between ztunnels travels over an HTTPS CONNECT tunnel.

Waypoint proxy Provides L7 telemetry and authorization policies at the HTTP level.

Cilium: There's no always-on second proxy hop for L4 traffic. eBPF is the L4 mesh layer, running in-kernel rather than as a sidecar or per-node daemon process. Envoy only gets invoked, flow-by-flow, exactly when L7 visibility/policy is actually required.

8.3 Traffic management / deployment strategies

In a microservices graph, services call each other extensively, so a single failing service doesn't fail in isolation — its callers start failing too, and their callers after that. This cascading failure pattern is the core risk a service mesh's traffic-management features are designed to contain

GAMMA

The GAMMA Initiative (Gateway API for Mesh Management and Administration) extends the Kubernetes Gateway API — originally built for north/south ingress traffic — to also govern east/west (service-to-service) traffic inside the mesh, via the same HTTPRoute resource. It's increasingly supported across service meshes, and it's the mechanism that makes some of the canary routing below possible (e.g. header-based routing uses HTTPRoute rules under the hood).

Rolling Deployment

Replaces old instances with new ones in batches. Ensures zero downtime and doesn't require a duplicate environment — but both versions coexist temporarily while the rollout is in progress.

Recreate / Big-bang Deployment

Shuts down the entire old version before deploying the new one. Simple, but causes inevitable downtime since there's a gap with nothing running.

Canary Deployment

Releases the new version to a small, select group of users before a full rollout — minimizing blast radius and allowing real-world testing.

- **With plain Kubernetes Services:** traffic is always split evenly across all pods behind a Service. To get a specific weight (e.g. v1 getting less traffic than v2), you have to create additional pod replicas as a workaround — there's no other way to weight or selectively stop traffic to specific pods.
- **With a service mesh:** arbitrary traffic weights are possible directly, no extra pods needed. Routing can also be conditioned on headers rather than just weight, letting you target specific test traffic into the canary precisely.

Blue/Green Deployment

The environment running the previous version is kept alive rather than torn down. Traffic is switched all at once (100%/0%) rather than gradually. This allows a fast fallback to the old deployment even after a full switch, and can be combined with canary deployments (e.g. canary first, then a full blue/green cutover once confidence is high).

Dark Launching / Shadow Deployment

Mirrors live traffic to the new version without it affecting the actual user-facing response — the real response still comes from the old version, but the new version processes a copy of the traffic in parallel. This allows silent testing under real load with zero user-facing risk.

8.4 Security

Motivation: all pods sit on a flat network by default, so anything can talk to anything — service mesh adds access control on top.

Two policy types:

- **Authentication policies** — define who can talk to whom (service identity)
- **Authorization policies** — fine-grained access control via RBAC, can be as granular as HTTP method/path (GET/POST/PATCH/DELETE) per service

Authentication splits into:

- **Service-to-service:** mutual TLS (mTLS)
- **User-to-service:** JWTs, OpenID Connect

Other security functions: transparent encryption of all inter-service traffic (mTLS / WireGuard / IPSec), automatic certificate/key issuance and rotation.

Key Takeaways

1. Microservices solve scalability/modularity but create distributed-systems problems
2. A service mesh = data plane (proxies) + control plane (policy distribution)
3. Sidecars are invasive and resource-inefficient, motivating a sidecarless design
4. L7 is the one layer that can't be fully pushed into the kernel
5. Cascading failure is the core risk that traffic-management features defend against
6. Plain Kubernetes Services can't do weighted or conditional traffic splitting — a service mesh can
7. All pods sit on a flat network by default, so anything can talk to anything. Necessitating the need for: Authentication, Authorization, Traffic encryption and Certificate/Key rotation

9. Kubernetes Observability

Learning Objectives

- No dedicated "Learning Objectives"

Kubernetes doesn't provide observability out of the box: historical metrics aren't persisted by default, `kubectl logs` only shows recent output for running containers, logs vanish when a container dies, and there are no native dashboards or alerts. Since distributed systems can fail at the node, pod, container, or application level, closing this gap is core to both DevOps and SRE practice.

	Monitoring	Logging
Nature	Quantitative	Qualitative
Captures	Trends over time	Detailed event records
Examples	Utilization (memory, disk), counters (requests), rates (error rate)	Stack traces, system messages, business events

	Push (Agent-based)	Pull (Agentless)
Mechanism	Agent installed on the monitored system sends signals to the collector	Collector queries the monitored system over an agreed protocol (SNMP, ICMP, HTTP, etc.)
Advantage	Observed system decides when to send data	Collector decides when it is ready to query the system
Disadvantage	Running an agent is not possible everywhere	Observed system has to be online all the time

9.1 Monitoring with Prometheus

Prometheus

Prometheus is a open-source metric collector and time-series DB. It's **pull-based** (scrapes `/metrics` endpoints over HTTP). It was originally developed at SoundCloud, and donated to the CNCF in 2016.

It provides a powerful but simple data model to store metrics and uses its own query Language (PromQL).

It is suitable for using with complex workloads since it can collect millions of metrics/second.

In a Kubernetes context, Prometheus scrapes targets such as kubelets, kube-state-metrics, and exporters. The Prometheus Operator allows scrape configuration to be managed declaratively via Kubernetes custom resources, while Grafana sits on top as the dashboarding and visualization layer.

Architecture

The Prometheus server itself consists of three core components:

- **Retrieval component** that scrapes targets
- **TSDB** (time-series database) that stores the collected data locally on disk (HDD/SSD)
- **HTTP server** that exposes the data for querying.

Target

An endpoint Prometheus scrapes for metrics — e.g. a kubelet, kube-state-metrics, or an exporter. Targets can be defined statically or dynamically (via the Prometheus Operator on Kubernetes).

Exporter

A mechanism for third-party software that does not natively expose a `/metrics` endpoint. It uses the software's own interface (SQL, filesystem, custom protocol, etc.) to gather information and exposes it in a format Prometheus can scrape. Prometheus then requests metrics from the exporter instead of from the app directly. Use an exporter whenever you can't instrument the code itself — examples include PostgreSQL, Redis, Kafka, and JIRA exporters.

PromQL

PromQL is Prometheus's functional query language, used to select and aggregate time-series data in real time — for building graphs/visualizations and for defining alerts. An expression can evaluate to one of four types:

	Description
Instant vector	Set of time series, each with a single sample, all sharing the same timestamp
Range vector	Set of time series, each with a range of data points over time
Scalar	Simple numeric floating-point value
String	Simple string value (currently unused)

```
# Instant Vector Selector
http_requests_total{environment=~"staging|testing", method!="GET"}

# Range Vector Selector
http_requests_total{job="prometheus"}[5m]

# Time Durations
[5h] [1h30m] [5m] [10s]

# Offset Modifier (positive = backwards in time!)
rate(http_requests_total[5m] offset 1w)

# @-Modifier
sum(http_requests_total{method="GET"} @ 1609746000)
```

Time durations (e.g. `[5m]`, `[1h30m]`), the offset modifier, and the `@` modifier allow queries to look at specific or shifted time windows.

Each metric is identified by a name plus an unordered set of key-value pairs called labels. PromQL allows the aggregation across labels to analyze metrics

- Per process

- Per datacenter
- Per service
- Per any other label

Metrics Format:

Prometheus exposes metrics in a text-based exposition format that is both human-readable and easy to parse for machines. The basic format is:

```
<metric_name>{<label_name>=<label_value>, ...} <metric_value> [<timestamp>]
```

The timestamp is optional — it's only needed when ingesting historical data or during remote-write.

Example:

```
http_requests_total{method="GET", handler="/api/status", status="200"} 1427 1776870361000
node_filesystem_avail_bytes{mountpoint="/home"} 207583686656
```

Prometheus supports different metric types, for example:

- **Counter** — only increases or resets to zero (e.g. requests served, errors, completed tasks)
- **Gauge** — a single numerical value that can go up or down (e.g. current memory usage, concurrent requests)
- **Histogram** — samples observations into buckets
- **Summary** — also samples observations, but additionally provides a total/sum

Scaling

Since a single Prometheus server is standalone, scaling requires additional patterns:

- **Federation** — a hierarchical setup, e.g. a per-region Prometheus scraping selected metrics from per-datacenter Prometheus instances
- **Thanos** — the most popular federation solution; runs as a sidecar and adds global queries across instances, long-term storage, and deduplication
- **Grafana Mimir** — a horizontally scalable reimplementation of Prometheus with separate components (ingester, querier, etc.), supporting multi-tenancy at very large scale; uses the remote-write (push) model exclusively

9.2 Visualization & Alerting (Grafana)

Grafana

Grafana is a general-purpose data visualization application with a strong emphasis on aesthetics. It works with a wide range of data sources — Prometheus among them — and can pull data from many integrations.

Querying

Grafana sits in front of the collector(s) as a query frontend, in this case Prometheus. Grafana integrates well with identity providers (OAuth — generic, Entra ID, GitHub, etc., as well as SAML and LDAP), making it the natural place to control access. Its “Explore” feature lets you browse data sources and build queries interactively, which is useful for debugging or exploration and can be shared with other team members.

Dashboards

Dashboards give a visual summary of system health, typically through panels like CPU, memory, network, and pod restarts. Grafana runs a public dashboard repository (grafana.com/grafana/dashboards) for importing ready-made dashboards, and many open-source projects also commit their dashboards as JSON files directly to their repositories (e.g. the FluxCD monitoring example), making those easy to import as well.

Design tips for building good dashboards:

- **Most important data to the top** — a dashboard is for humans; a status light is more effective than a raw number
- **Plan for other data sources** — use the “Data Source” variable type so the dashboard stays usable as new sources are added later
- **Simplicity** — a fully-loaded dashboard looks impressive, but the actual goal is spotting when something is wrong
- **Work with the team** — share dashboards early so others working the same problems benefit too

Alerting

	Prometheus Alertmanager	Grafana Alerts
Integration	Part of the Prometheus ecosystem	Integrated into Grafana, which becomes an alert hub
Good	Separates visualization and alerting; GitOps friendly	All Grafana data sources can trigger alerts; alert creation is tied to dashboards; can connect to external Alertmanagers
Bad	Works only for Prometheus	Not GitOps friendly; alert routing for larger infrastructure not yet optimal

Alertmanager flow:

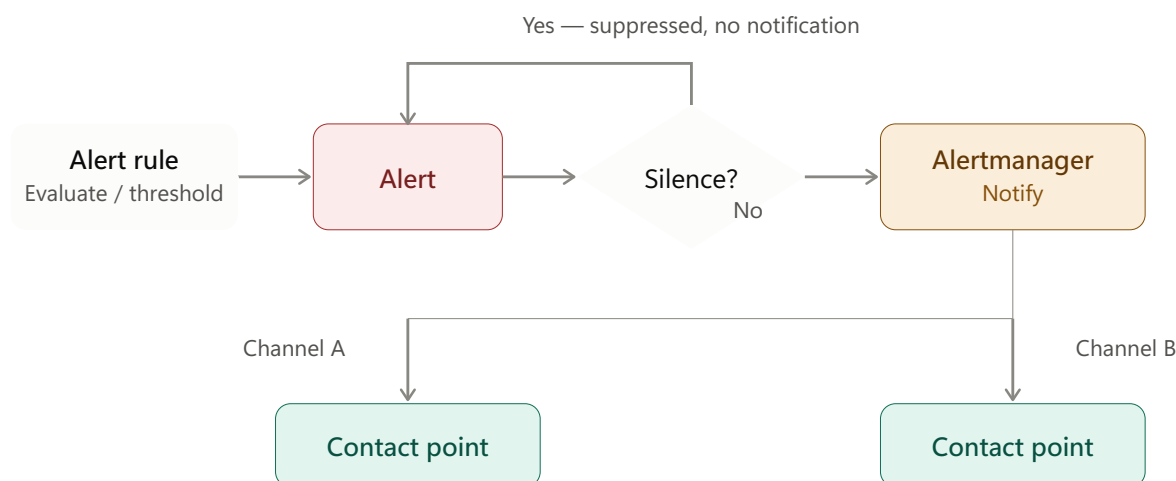


Figure 7: Alertmangager Flow

Good practices for alerting:

- Test your alerts — catch issues like a missing SMTP config or an expired Teams token before you need the alert
- Build up slowly, working down your priority list — avoid alert fatigue at all cost
- Warning $\neq$ Alert
 - “A Let’s Encrypt certificate will expire in 45 days” is a warning, not an urgent alert
 - “CPU utilization of worker-01 is at 75%” sits in between - take a closer look
 - “it’s the third day of the month and your Azure budget has reached 200% of planned” is a clear alert

9.3 Logging with Loki

In Kubernetes each container writes its logs to stdout/stderr. These logs are ephemeral and tied to the container’s lifecycle — `kubectl logs` reads them directly from the container runtime, but once a container dies, its logs are gone with it. This is why centralized logging is required for any kind of historical analysis.

Since logs are mostly unstructured text rather than structured data, a centralized logging platform exists to provide a historical record of events across applications, systems, network devices, and other infrastructure components, along with full-text search. The typical use cases are troubleshooting, debugging, and forensic analysis.

Kubernetes logging approaches

There are three broad ways to get logs out of a cluster:

- **Sidecar containers** for log shipping — less common
- **DaemonSets** (Fluent Bit, Alloy, Promtail) — a pod runs on every node and collects logs for all workloads on that node
- **Log drivers** and **plugins** can add more advanced features on top

Best practice regardless of approach: use structured logs (JSON, machine-readable, with labels attached).

Cluster-level logging architectures These approaches map onto three architecture levels:

- **Node level** — a logging agent runs on every node via DaemonSet; works even if the container dies, though shipping is batched rather than strictly real-time
- **App level** — the application pushes logs directly to a backend itself
- **Sidecar** — a dedicated logging container runs alongside the app inside the same Pod
 - Within the sidecar approach, there are two distinct patterns:
 - **Logging agent** — the sidecar exposes logs to stdout, which get written to a shared log-file.log (with logrotate managing rotation); a separate logging-agent-pod reads that file and ships it to the logging backend
 - **Streaming** — the sidecar logs directly to the backend itself, with no intermediate file

Loki**Loki**

Loki is a horizontally scalable, distributed log aggregation system. Its key idea is that it indexes labels, not full text.

In a cloud-native environment labels come naturally, so this keeps storage efficient and operational cost low.

The trade-off versus general-purpose log aggregation systems: no true full-text search, since it’s heavily dependent on labels, but in exchange you get excellent compression and fast queries.

Loki storage concepts

Each log line stored in Loki has three parts:

- a timestamp
- a set of labels
- and the message text.

Only the timestamp and labels are indexed

The pipeline

Three components work together:

- **Alloy** — collects logs from Kubernetes resources and sends them to Loki
- **Loki** — ingests logs, processes labels, and prepares them for querying (indexing)
- **Grafana** — connects to Loki as a query frontend, providing dashboarding and alerting

LogQL

LogQL is Loki's query language, syntactically similar to PromQL. It splits into two kinds of queries:

	Log Queries	Metric Queries
Purpose	Filter and search logs	Generate metrics from logs
Output	Log lines	Time series showing a rate
Example	<code>{app="api"} = "error"</code>	<code>rate({app="log-generator"} = "176" [1m])</code>

Every LogQL query starts with a log stream selector (the `{...}` label matchers), optionally followed by a log pipeline for further filtering:

```
{ log stream selector } | log pipeline
```

Label filters — applied to the indexed labels themselves:

	Meaning
<code>==</code> or <code>=</code>	Equality
<code>!=</code>	Inequality
<code>></code> , <code>>=</code>	Greater than / greater than or equal
<code>and</code> , <code>or</code>	Chain multiple filter operations

Log line filters — applied to the unindexed message text:

	Meaning
<code> =</code>	Log line contains string
<code>!=</code>	Log line does not contain string
<code> ~</code>	Log line contains a match to the regex
<code>!~</code>	Log line does not contain a regex match

Alloy

Alloy

Alloy isn't limited to logs — it collects all types of telemetry. It's configured using its own Alloy language, which is similar to the HCL used in OpenTofu/Terraform, and ships with integrations for Azure, Docker, Kubernetes, EC2, and more. It's installable via Helm chart. In the logging path specifically, Alloy enriches logs with Kubernetes metadata and forwards them to Loki over HTTP.

Classically, DaemonSets were deployed on every node to tap directly into `/var/log` and collect from every Kubernetes component. Alloy can replicate this with its `loki.source.file` component (reading log messages from files) — but it can alternatively use `loki.source.kubernetes` to subscribe to logs via the Kubernetes API instead.

	Classic DaemonSet / <code>loki.source.file</code>	<code>loki.source.kubernetes</code>
Privileges	Often needs elevated/root access	Does not require root privileges
Deployment	Needs to run as a DaemonSet	Does not need to run as a DaemonSet
Resource cost	Lower CPU/network use	Uses more CPU, generates more network traffic

Alloy can also use `loki.source.kubernetes_events` to collect Kubernetes events themselves, and — like the classic approaches — supports filtering and relabeling logs before they're forwarded.

A minimal Alloy pipeline for shipping pod logs to Loki looks like this:

```
discovery.kubernetes "pods" {
  role = "pod"
}

discovery.relabel "pod_logs" {
  targets = discovery.kubernetes.pods.targets
  rule {
    source_labels = ["__meta_kubernetes_pod_node_name"]
    target_label  = "__host__"
  }
}

loki.source.kubernetes "pods" {
  targets      = discovery.relabel.pod_logs.output
  forward_to = [loki.write.default.receiver]
}

loki.write "default" {
  endpoint {
    url = "http://loki.monitoring.svc.cluster.local:3100/loki/api/v1/push"
  }
}
```

9.4 Tracing

Traces

Traces capture the path of a single request through a distributed system. A single trace is composed of multiple spans — each span starts when its corresponding context opens and ends when that context closes. Traces provide a request-level view that shows how services interact, which makes them very helpful for debugging microservices.

9.5 Wrap-Up

Monitoring and logging — best practices

- Use labels to add context to your metrics and logs
- Find out what data is actually important to you (the slides point to Google’s SRE book section on the four golden signals)
- Retain only what you need
- Avoid excessive verbosity
- Set meaningful alert thresholds — avoid alert fatigue
- Don’t log secrets!

Common pitfalls

- Too much logging increases storage and complexity
- Lack of alert tuning causes noise or missed issues
- Forgetting log rotation and archival strategies
- Relying only on one observability method

Alternative Stacks

	Components	Notes
EFK	Elasticsearch, Fluentd, Kibana	
PLG	Prometheus, Loki, Grafana	Lightweight, efficient, K8s-native
LGTM	Loki, Grafana, Tempo, Mimir	
OpenTelemetry	—	Unified telemetry solution

Key Takeaways

1. K8s doesn’t give you observability by default. No persisted historical metrics, ephemeral logs tied to container lifecycle, no native dashboards or alerting
2. Observability = how well you can infer system state; monitoring = actually collecting/storing/evaluating the signals (metrics, logs, traces)
3. Metrics (Prometheus) and logs (Loki) are architecturally similar by design
4. Loki’s “index labels, not full text” choice explains its storage efficiency and its lack of full-text search
5. Node-level DaemonSet agents survive container crashes but batch
6. Sidecars can stream in near real-time but lose data if the app crashes early or the connection drops
7. Tracing is the request-level complement to metrics/logs — it answers “how did this one request move through my services,” which neither metrics nor logs alone can show.

10. GitOps

Learning Objectives

- Define GitOps and explain its core principles (Declarativity, Version Control, Automated Delivery, Continuous Reconciliation).
- Contrast GitOps with traditional CI/CD push-based models.
- Compare the two leading implementations: Argo CD and Flux CD.
- Evaluate trade-offs to select the right tool for a specific scenario.

GitOps

Operating model where Git is the single source of truth for declarative infrastructure and applications, with automated reconciliation.

10.1 Principles

	Principle	Meaning
1	Declarative	The system's desired state is expressed declaratively
2	Versioned and Immutable	Desired state is stored in a way that enforces immutability, versioning, and a full history
3	Pulled Automatically	Software agents automatically pull the desired state from the source
4	Continuously Reconciled	Agents continuously observe actual state and attempt to apply the desired state

Side effects of following these principles: **documentation** (Git history is the audit trail) and **reproducibility**.

10.2 Traditional CI/CD vs. GitOps

Aspect	Traditional CI/CD	GitOps
Trigger	CI pushes to prod	Agent pulls from Git
Trust boundary	CI server trusted	Agent / Git repository trusted
Drift detection	Complex / manual	Automatic
Rollback	Complex / manual	Revert
Security	CI needs prod admin access	Agent needs Git read / prod admin access

10.3 Push vs. pull

Push-based

1. Pros

- Ease of use
- Well-known as part of CI/CD
- Fast deployment speed

— Cons

- Requires opening firewall to the cluster and granting admin access to external CI/CD
- Requires adjusting CI/CD pipelines for new environments
- Configuration drift is harder to manage

Pull-based

1. Pros

- Secure infrastructure — no need to open firewall or grant admin access externally
- Changes can be automatically detected and applied without human intervention
- Easier scaling of identical clusters

— Cons

- You need an agent
- Moderate deployment speed, depending on polling interval and trigger

10.4 Argo CD

Argo CD

Developed by Intuit (2018), CNCF Graduated (2021). Primary interface: Web UI + CLI. Philosophy: “Platform Engineering” approach. Best for: enterprises, multi-cluster setups, and teams needing a GUI.

Key features: visual dashboard (app health, sync status, history), hooks (pre-sync/post-sync for complex logic), multi-cluster support from a single UI, granular access control via SSO, built-in health checks.

Key resources:

- **Application** — the core resource: what (which repo), where (cluster/namespace), how (sync policy) to deploy
- **ApplicationSet** — a generator that creates multiple Applications from a template + list of inputs (e.g. one app per branch, - cluster, or environment)
- **AppProject** — logical grouping that restricts what can be deployed where, and who has access

Example:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
  namespace: argocd
spec:
  source:
    repoURL: https://github.com/org/repo.git
    targetRevision: HEAD
    path: manifests
  destination:
```

```

server: https://kubernetes.default.svc
namespace: production
syncPolicy:
  automated:
    prune: true
    selfHeal: true

```

10.5 Flux CD

Flux CD

Developed by Weaveworks (2018), CNCF Graduated (2022). Primary interface: CLI + Git (Kubernetes-native). Philosophy: “CLI first” approach. Best for: developers preferring CLI, Helm apps, “Gitless” GitOps.

Key features: the **GitOps Toolkit** (modular components), deep native Kustomize integration, “Gitless” support (storing artifacts in OCI registries or S3 buckets instead of Git), a lighter resource footprint than Argo CD, and no central UI by default (CLI-focused, though a separate UI project exists).

Key resources:

- **Sources** — store resource definitions: `GitRepository`, `OCIRepository`, `Bucket`, `HelmRepository` (containing `HelmChart` resources)
- **Kustomize** — applies stored manifests via `Kustomization`
- **Helm** — applies Helm charts via `HelmRelease`

There are two different `Kustomization` resources — `kustomize.config.k8s.io` (the underlying Kustomize feature itself) vs. `kustomize.toolkit.fluxcd.io` (the Flux CRD that tells Flux to look for and apply a `Kustomization`). Same name, different API groups, different purpose.

Example:

```

apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: webapp
  namespace: default
spec:
  interval: 1m
  url: https://github.com/org/repo.git
  ref:
    branch: main
---
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: webapp
  namespace: default
spec:
  interval: 10m
  sourceRef:
    kind: GitRepository
    name: webapp
  path: ./kustomize
  prune: true
  wait: true

```

10.6 GitOps Workflow Patterns

- **App of Apps pattern** (Argo-flavored): a single root Argo **Application** syncs from a repo that itself contains references to multiple child **Application** resources, each pointing at its own repo — a tree of Applications managing Applications.
- **Monorepo workflow** (Flux-flavored): a structured single repository —
 - `./clusters/<cluster>/` — Flux itself plus root Kustomizations (e.g. `flux-system-extra`, `apps`)
 - `./clusters/flux-system-extra/` — shared resources across all clusters (Helm repositories, CRDs)
 - `./apps/base/` — shared applications (controllers, configuration, common repositories)
 - `./apps/<cluster>/` — per-cluster application overlays

10.7 Secrets Management

	Encrypted secrets	References to secrets
Tools	Bitnami Sealed Secrets; Mozilla SOPS	ExternalSecrets; Kubernetes Secrets Store CSI driver
Idea	Encrypt the secret itself so it's safe to commit to Git	Commit only a reference/pointer; the real secret lives in an external system

Key Takeaways

1. Push-based pipelines have no defense against someone bypassing them entirely
2. Pull-based agents don't just deploy once, they keep checking and correcting
3. Drift detection and rollback become automatic: the agent is always comparing against Git, and "rollback" just means "revert in Git."
4. Argo CD: UI-first, platform-engineering oriented, strong multi-cluster/SSO story
5. Flux CD: CLI-first, Kubernetes-native, modular toolkit, lighter footprint
6. App of Apps & monorepo-with-overlays solve the same multi-cluster/multi-env problem differently
7. Either encrypt the secret so it's safe to commit (Sealed Secrets, SOPS) or commit only a reference and keep the real value elsewhere (ExternalSecrets, CSI driver)